

KTDS

설계서

TEST_SHOPPING_MALL

API 설계서 26

버전	v1.0
작성일	2026-09-20
작성자	이주형

대외비

목차

API 설계서 26건

- INF-MBR-001 회원가입 인증코드 발송
- INF-MBR-002 가입 요청(인증코드 검증 포함)
- INF-MBR-003 로그인
- INF-MBR-004 로그아웃
- INF-MBR-005 자동 로그인 토큰(리프레시)
- INF-MBR-006 비밀번호 재설정 코드 요청
- INF-MBR-007 비밀번호 재설정 확정
- INF-MBR-008 배송지 CRUD
- INF-MBR-010 api members grades
- INF-ORD-001 회원 목록 조회
- INF-ORD-002 회원 상세 조회
- INF-ORD-003 주문 목록 조회
- INF-ORD-004 주문 상세 조회
- INF-ORD-005 주문 생성
- INF-ORD-006 주문 취소
- INF-ORD-007 주문 배송 목록 조회
- INF-ORD-008 판매중 상품 목록 조회
- INF-ORD-009 상품 단건 조회
- INF-ORD-010 장바구니 담기
- INF-ORD-011 장바구니 조회
- INF-ORD-012 장바구니 수량 변경
- INF-ORD-013 장바구니 품목 삭제
- INF-ORD-014 장바구니 체크아웃
- INF-ORD-015 주문 목록 CSV 내보내기
- INF-ORD-016 api zipcodes
- INF-ORD-017 shop spa static serving

개정 이력

버전	일자	내용	작성자
v1.0	2026-09-20	최초 작성	

API 설계서

INF-MBR-001 회원가입 인증코드 발송

INF-MBR-001: POST

/api/members/signup/verification-codes — 회원 가입 인증코드 발송

> ****개요:**** 이메일 또는 휴대폰번호를 받아 6자리 인증코드를 생성·저장(5분 유효)하고 "발송"한다(이 랩은 실제 발송 게이트웨이가 없어 로그로 대체). 인증코드 확인 + 회원 생성(가입완료)은 별도 API(INF-MBR-002, FUNC-member-003)의 책임이며 이 API 범위 밖이다.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberSignupController.java:38-42`

요청

Method: POST

Path: /api/members/signup/verification-codes

Content-Type: application/json

인증: 없음(무인증) — 신규 가입자는 아직 API 키가 없으므로 `ApiKeyAuthFilter` 화이트리스트(`MEMBER\_SIGNUP\_VERIFICATION\_CODE\_PATH`)에 등록되어 있다.

파라미터	위치	타입	필수	설명
target	Body	string	Y	인증코드를 받을 대상 — 이메일 주소 또는 휴대폰번호(하이픈 없이 숫자만). 최대 100자 (DB 컬럼 `target VARCHAR(100)` 정합). 형식으로 채널(EMAIL/SMS)을 자동 판별한다

응답 (200 OK)

```
{
  "channel": "EMAIL",
  "target": "user@example.com",
  "expiresInSeconds": 300
}
```

`channel`: 판별된 발송 채널 — `EMAIL` 또는 `SMS`

`target`: 요청받은 값의 에코(trim 처리됨) — 개인정보 누출 아님(요청자 본인이 보낸 값)

`expiresInSeconds`: 코드 유효시간(고정값 300) — 코드 자체는 응답에 담기지 않는다(로그에도 마스킹)

비즈니스 규칙

인증코드는 6자리 숫자, 발급 시점부터 5분(300초) 유효(`MemberSignupService.EXPIRES\_IN\_SECONDS`)
 채널 자동판별: 이메일 정규식 매칭 → `EMAIL`, 국내 휴대폰번호 패턴(`01[016789]` + 7~8자리, 하이픈 없음) 매칭 → `SMS`, 둘 다 아니면 400
 레이트리미트(동일 `target` 기준, 채널 무관): 60초 쿨다운 + 하루 5회 상한. **거부된 요청은 quota를 소비하지 않는다** — 원자 UPSERT의 조건부 갱신이라 `daily\_count`/`last\_token`이 갱신되지 않으면 그 요청은 카운트에 반영되지 않는다
 판정 방식(round5/round6, 사람 결정): 요청마다 생성한 UUID 토큰을 조건부로 `last\_token`에 기록한 뒤 재조회한 값이 자기 토큰과 같으면 허용 — JDBC affected-rows나 datasource 전역 설정(`useAffectedRows`)에 더 이상 의존하지 않는다(그 설정이 이 FUNC 밖 `ProductDao.decreaseStock`의 반환값 시맨틱까지 바꿔 `POST /api/orders`를 200→409로 회귀시킨 적이 있어 제거됨)
 발송은 시뮬레이션(로그 1줄, 개인정보 마스킹 — 앞 2자 제외 `\*` 처리) — 실제 이메일/SMS 게이트웨이 미연동. 코드 값은 응답·로그 어디에도 노출하지 않는다
 레거시 주의: `MEMBER\_SIGNUP\_VERIFICATIONS.requested\_at`/`previous\_requested\_at`/`daily\_count` 컬럼은 round3 잔재(deprecated) — round4부터 앱이 더 이상 쓰지 않는다(카운터는 `MEMBER\_SIGNUP\_RATE\_LIMITS`로 완전히 이전됨).
 재발송 시 시도 횟수·소비 표시 리셋(SR-295): 같은 채널·타깃 행에 재발송하면 새 6자리 코드·만료시각과 함께 `attempt\_count=0`, `consumed\_at=NULL`, `verified\_at=NULL`로 함께 리셋한다 — 잠긴(`attempt\_count>=5`) `target`이 재발송을 받으면 새 코드로 정상 검증 가능해진다(INF-MBR-002 `MBR-4093` 회복 경로). `attempt\_count`/`consumed\_at`/`verified\_at`의 판정·소비는 여전히 INF-MBR-002(FUNC-member-003)가 소유한다 — 이 API는 재발송 시 초기화만 한다.
 만료행 정리(purge)는 이 요청 경로에 없다 — `MemberSignupMaintenanceScheduler`(`@Scheduled`, 10분 주기 배치)가 `purgeExpiredCodes`/`purgeOldRows`를 별도로 호출한다

트랜잭션 순서

1. `MEMBER\_SIGNUP\_RATE\_LIMITS`에 원자 UPSERT(`touchRateLimit`) — 쿨다운·일일상한 조건을 동시에 만족할 때만 `daily\_count`/`last\_requested\_at`/`last\_token`을 이번 요청 토큰으로 갱신(단일 문장, autocommit — `@Transactional` 없음. round4에서 의도적으로 제거해 `purgeExpired`와의 락 경합·데드락을 없앴다)
2. 같은 `target`/`day\_key`를 재조회(`selectRateLimit`)해 `last\_token`이 내 토큰과 같은지 확인 — 다르면 여기서 429로 즉시 응답하고 코드는 생성하지 않는다
3. 판정을 통과한 경우에만 `MEMBER\_SIGNUP\_VERIFICATIONS`에 원자 UPSERT(`writeCode`)로 6자리 코드·만료시각을 기록한다(1단계와 별개의 autocommit 문 — 하나의 트랜잭션으로 묶여 있지 않다). 재발송(기존 행 갱신)이면 같은 문장에서 `attempt\_count=0`, `consumed\_at=NULL`, `verified\_at=NULL`도 함께 리셋한다(SR-295) — 신규 행(INSERT)은 DDL 기본값이 이미 이 값들이라 리셋 대상이 없다

오류 응답

응답 봉투는 모두 `{ "code": "...", "message": "..." }` 형태(`MemberSignupExceptionHandler`).

코드	사유	발생 조건

400	MEMBER_TARGET_INVALID	`target`이 비었거나 100자를 초과, 또는 이메일·휴대폰번호 형식에 둘 다 불일치
429	MEMBER_VERIFY_COOLDOWN	동일 `target` 재발송을 60초 이내에 재요청(쿨다운 미경과)
429	MEMBER_VERIFY_DAILY_LIMIT_EXCEEDED	동일 `target`이 오늘 이미 5회 요청을 완료
500	MBR-5000	DB 계층 예외(`DataAccessException`) — 정제된 메시지만 응답, 경로·SQL·커넥션 정보는 노출하지 않으며 원본은 서버 로그에만 남는다(무인증 엔드포인트라 정보 노출 위험이 특히 큼)

참조 테이블

`MEMBER\_SIGNUP\_VERIFICATIONS`

`MEMBER\_SIGNUP\_RATE\_LIMITS`

curl 예시

```
curl -X POST /api/members/signup/verification-codes \
  -H "Content-Type: application/json" \
  -d '{"target": "user@example.com"}'
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-15	SR-295	#1	재발송(writeCode) 시 attempt_count/consumed_at/verified_at을 함께 리셋하도록 UPSERT SET절 확장 — MBR-4093 회복 경로	shop-api@45e21e8

INF-MBR-002 가입 요청(인증코드 검증 포함)

> [반영: FUNC-member-003] 2026-09-12

INF-MBR-002: POST /api/members/signup — 가입 요청(인증코드 검증 포함)

> ****개요:**** 이메일 또는 휴대폰번호 + 인증코드(6자리) + 비밀번호·이름·마케팅 수신 동의를 받아 회원을 생성한다. 인증코드 "발송"은 별도 API(INF-MBR-001)의 책임이지만, 코드 ****검증****(대조·소비)은 발송 쪽이 아니라 이 API 자신이 수행한다 — `MEMBER\_SIGNUP\_VERIFICATIONS` 행은 INF-MBR-001이 써 둔 것을 이 API가 조건부 UPDATE로 대조·소비할 뿐, 그 테이블을 소유한 파일은 건드리지 않는다. round1~4에 걸친 QA 재작업으로 "① 코드 검증(비트랜잭션) → ② 존재 판정(비트랜잭션, 안내용) → ③ 가입(트랜잭션, 별도 스프링 빈)"의 3단계 순서가 사람이 직접 지정한 것으로 고정됐다 — 이 순서 자체가 보안 요건(존재 오라클 방지)이므로 임의로 바꾸면 안 된다.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberRegistrationController.java:34-40`

요청

Method: POST

Path: /api/members/signup

Content-Type: application/json

인증: 없음(무인증) — 신규 가입자는 아직 API 키가 없으므로 `ApiKeyAuthFilter` 화이트리스트(`MEMBER\_SIGNUP\_REQUEST\_PATH`, 정확 일치)에 등록되어 있다. ****레이트리미트 없음****(아래 오류 표·비즈니스 규칙 참고).

파라미터	위치	타입	필수	설명
target	Body	string	Y	인증코드를 받은 대상 — 이메일 주소 또는 휴대폰번호(하이픈 없이 숫자만). 값 자체로 채널(EMAIL/SMS)을 자동 판별(INF-MBR-001과 동일 정규식을 이 서비스에 복제해 사용)
code	Body	string	Y	INF-MBR-001이 발송한 6자리 인증코드. 이 API가 직접 대조·소비한다
password	Body	string	Y	8~64자, 영문+숫자 포함(정규식 위반 시 400)
name	Body	string	Y	회원 이름, 최대 50자(빈 값·공백만·50자 초과 시 400 — `MEMBERS.member_name VARCHAR(50)` 정합)

marketin gOptIn	Bo dy	bool ean	N	마케팅 수신 동의. 생략(null) 시 'false'로 처리
--------------------	----------	-------------	---	-----------------------------------

****서버 내부 검증 순서(그대로 구현된 순서 — 응답 코드가 이 순서에 의존한다):****

1. `target` 공백/누락 → 409 `MBR-4091` (형식이 성립 않으면 인증 기록도 존재할 수 없다는 논리로 "인증 필요"에 수렴)
2. `target` 형식이 이메일·휴대폰 정규식 둘 다 불일치 → 409 `MBR-4091` (동일 논리)
3. `name` 공백/누락/50자 초과 → 400 `MBR-4001`
4. `password` 규칙 위반 → 400 `MBR-4001`
5. (STEP 1) 코드 검증 실패 → 409 `MBR-4091` 또는 `MBR-4093`
6. (STEP 0) 코드 검증 성공 후에만 중복 판정 → 409 `MBR-4092` (+`login\_url`) 또는 `MBR-4094`
7. BCrypt 해시 계산 → (STEP 2) 가입 트랜잭션 → 성공 시 201

응답 (201 Created)

```
{
  "memberId": "M-0053",
  "channel": "EMAIL",
  "target": "user@example.com"
}
```

`memberId`: 신규 채번된 회원 ID, `M-` + `%04d` 포맷(`ID\_SEQUENCES` 원자 채번, `MEMBER\_ID` 시퀀스)

`channel`: 판별된 채널 — `EMAIL` 또는 `SMS`

`target`: 요청받은 값의 에코(trim 처리됨)

응답에 이메일·비밀번호 해시·마케팅 동의값 등은 포함하지 않는다

비즈니스 규칙

****3단계 흐름과 트랜잭션 경계(사람이 직접 지정, round3~4 최종 확정 — 임의 변경 금지):****

****STEP 1 — 코드 검증****(`MemberRegistrationService#verifyCode`, 비트랜잭션):

`markVerifiedIfCodeMatches` 조건부 UPDATE(`channel`+`target`+`code`+미만료+`consumed\_at` IS NULL+`verified\_at` IS NULL+`attempt\_count` < 5)로 검증. 0행이면 `incrementAttemptCount` (SR-298 — `channel`+`target`+`consumed\_at` IS NULL+`verified\_at` IS NULL+`attempt\_count` < 5` 조건부 UPDATE로 증가와 상한판정을 한 문장에 원자화, read-modify-write 아님)를 시도한다. 이 원자 UPDATE가 1행이면 정상 증가 후 409 `MBR-4091`. \*\*0행이면(사유가 상한 도달 하나가 아니다 — 이미 소비·이미 인증된 행이거나 매치 행 자체가 없어도 0행이다) 그 순간만 `selectAttemptCount`로 현재 값을 읽어 원인만 재분기\*\*한다: `attempt_count` >= 5(상한 도달)면 409 `MBR-4093`, 그 밖의 모든 0행 사유는 409 `MBR-4091` (재증가하지 않는다 — 증가는 이미 원자 UPDATE에서 끝났다). 이 읽기는 경합 판정이 아니라 오류 코드 표시용일 뿐이며, "5회 도달 후 카운터가 더 늘지 않는다"는 성질은 이제 이 UPDATE의 `attempt_count` < 5` 조건절 자체

가 보장한다(select→분기→update 패턴 아님). 각 UPDATE는 autocommit 개별 문장 — 이후 어떤 예외로도 롤백되지 않는다.

****STEP 0 — 존재 판정****(`precheckDuplicate`, 비트랜잭션 읽기, ****STEP 1 성공 뒤에만 호출****): `MEMBERS`를 `email`/`phone\_norm`으로 직접 조회해 이미 있으면 이메일→409 `MBR-4092`(`+login\_url`), 휴대폰→409 `MBR-4094`. 안내용일 뿐 최종 보장은 STEP 2의 UNIQUE 위반 캐치다.

****STEP 2 — 가입****(`MemberSignupCompletionWriter#completeSignup`, `@Transactional`, 이 메서드만 갖는 별도 스프링 빈): ①`ID\_SEQUENCES` 원자 채번(`touchMemberIdSeq`+`selectLastMemberIdSeq`, 같은 커넥션 필수 — `LAST\_INSERT\_ID()` 세션 스코프) ②`MEMBERS` INSERT

③`MEMBER\_SIGNUP\_VERIFICATIONS.consumed\_at` UPDATE. ****이 세 문장만**** — 별도 빈으로 분리한 이유는 self-invocation(같은 클래스 안 호출은 프록시를 거치지 않아 `@Transactional`이 적용되지 않는 Spring AOP 함정)을 원천 차단하기 위함(과거 라운드에서 이 함정으로 시도횟수 카운터가 롤백되는 결함이 있었다).

`MemberRegistrationService#signUp` 자체는 `@Transactional`이 전혀 없는 순수 오케스트레이션 메서드다.

****존재 오라클 방지(사람이 명시 승인한 보안 우선 트레이드오프)****: STEP 1(코드 검증)이 STEP 0(존재 판정)보다 항상 먼저 실행된다. 코드 검증에 실패하면(오답·만료·미요청·이미 소비/인증됨·시도상한 도달 등 ****어떤 사유든****) STEP 0은 호출조차 되지 않는다 — 즉 ****코드 검증 실패는 그 target의 가입 여부와 무관하게 항상 동일한 409 `MBR-4091`****(같은 문구, 존재 여부에 따른 추가 조회가 없어 응답 시간대도 갈리지 않음)이다. 코드를 받지 못한 요청자는 이 target의 가입 여부를 알아낼 방법이 없다.

****부작용(사람이 명시 수용, round4 — SR-295로 발생 조건 축소)****: "이미 가입 완료된 target"이 ****재발송 없이**** 예전 코드로 재시도하면(그 코드의 `consumed\_at`이 이미 채워져 있으므로) STEP 1 자체가 실패해 409 `MBR-4092`가 아니라 ****409 `MBR-4091`****을 받는다. 자가치유는 있다 — 코드 만료(5분) + 정리배치(`MemberSignupMaintenanceScheduler`, 10분 주기)가 행을 지우거나, ****재발송을 받으면 즉시****(SR-295, `consumed\_at`이 리셋되므로) 다음 시도에서 정상적으로 `MBR-4092`(`+login\_url`)가 나온다. 실사용 영향은 "가입 직후 같은 날, 재발송 없이 재시도" 구간으로 한정된다.

****코드값 의미****:

`attempt\_count` >= 5: 해당 (channel,target) 코드는 잠금 상태 — 정답이어도 더 이상 매치되지 않음(409 `MBR-4093`). ****회복 경로(SR-295)****: 재발송(`writeCode`, INF-MBR-001 소유)이 같은 행의 `attempt\_count`를 0으로, `consumed\_at`/`verified\_at`을 `NULL`로 리셋하므로, 상한에 걸린 target도 재발송을 받으면 새 코드로 STEP 1 조건(`attempt\_count` < 5)을 다시 만족해 정상 검증 가능해진다. 이 STEP(STEP 1 SQL-조건식·순서)은 손대지 않았고, 선행 상태만 바뀐다.

****병렬 버스트 우회 차단(SR-298)****: `incrementAttemptCount`의 `attempt\_count` < 5` 조건절이 단일 PK(channel,target) 행 락 위에서 재평가되므로, 동시에 도착한 오답 요청들도 이 한 문장의 짧은 수명 동안만 직렬화된다 — 상한(5)을 넘겨 증가할 수 없다(이전에는 "현재값 읽기 → 상한 미만이면 별도 증가" 순서라 두 문장 사이 레이스로 5를 초과해 증가할 수 있었다). 0행 반환 시의 `selectAttemptCount` 읽기는 이 보장과 무관하다 — 그 시점엔 증가 여부가 이미 원자적으로 확정된 뒤이며, 오류 코드(4091 vs 4093) 표시에만 쓰인다. 부가로, 이 읽기가 있을 때(verification 행 없음) 3문장 / 없을 때(행 있고 1행 매치) 2문장으로 문장 수가 갈리지만, 어느 쪽도 `MEMBERS` 테이블을 조회하지 않아 존재 오라클 방지에는 영향이 없다(관측 가능한 응답 시간대 차이는 verification 테이블 PK 조회 1회 수준 — 회원 존재 신호를 만들지 않는다). `consumed_at` IS NOT NULL: 코드가 이미 가입에 사용됨 — 같은 코드로 재검증 시도는 항상 0행(재사용 차단).

`verified\_at`: STEP 1이 세팅하는 검증 시각. STEP 0에서 예외가 나도(트랜잭션이 아니므로) 롤백되지 않고 커밋된 채 남는다.

****login_url 필드****: 이 랩에는 실제 로그인 화면 라우트가 없다(grep 0건) — `"/login"`은 자리표시(placeholder) 값이며 실재하지 않는 경로다. MBR-4092 응답에만 채워진다.

****비밀번호 해시 시점****: BCrypt 계산은 STEP 1(코드 검증) 이후, STEP 2(가입 트랜잭션) 진입 직전에 수행한다 — 무인증 엔드포인트에서 시도 상한에 걸리는 요청까지 매번 BCrypt(수십~백 ms)를 돌리지 않기 위함.

****미분류(PK) 충돌 재시도****: STEP 2에서 `DuplicateKeyException` 발생 시 메시지에 실린 제약 이름으로 ``uq_members_email``→4092, ``uq_members_phone_norm``→4094로 분기하고, 둘 다 아니면(주로 PK 충돌) 완전히 새 트랜잭션으로 1회만 재채번 재시도한다 — 재시도도 실패하면 rethrow하여 일반 500 ``MBR-5000``으로 떨어뜨린다(업무 409로 오분류하지 않음).

****레거시/미해결 주의****: ① ``MemberSignupCompletionDao#selectVerifiedAt``/``#selectConsumedAt``는 테스트/조회 전용이며 운영 경로에서 호출되지 않는 죽은 코드로 남아 있다. ② 탈퇴 회원(``del_yn`='Y'`)의 email/phone\_norm이 UNIQUE 인덱스를 계속 점유해 재가입이 영구 불가하다(사전 판정도 ``del_yn``을 보지 않아 UNIQUE와 판정을 의도적으로 일치시킴) — 이 API 범위 밖, 백로그. ③ 이 엔드포인트에는 어떤 레이트리밋도 없다(존재 오라클은 막혔지만, 제3자가 "가입 진행 중인" target의 시도 5회를 대신 소진시켜 그 target의 가입을 일시 방해하는 경로는 남아 있다 — 일일 발송 상한 5회로 영향은 제한적).

트랜잭션 순서

- (비트랜잭션, autocommit) STEP 1 — ``MEMBER_SIGNUP_VERIFICATIONS`` 조건부 UPDATE(``markVerifiedIfCodeMatches``). 실패 시 원자 조건부 UPDATE(``incrementAttemptCount``, ``attempt_count < 5`` 포함 — SR-298 원자화)로 증가+상한판정을 한 문장에서 시도. 그 UPDATE가 0행이면(사유 다양 — 상한 도달/이미 소비/이미 인증/행 없음) ``selectAttemptCount`` 읽기 1회로 오류 코드만 재분기(재증가 없음). 즉시 응답 종료(이후 단계 진행 안 함).
- (비트랜잭션, 읽기) STEP 0 — ``MEMBERS``를 ``email``/``phone_norm``으로 조회(``selectMemberIdByEmailOrPhoneNorm``). 존재하면 즉시 응답 종료(이후 단계 진행 안 함).
- (트랜잭션 시작, ``MemberSignupCompletionWriter#completeSignup``) ① ``ID_SEQUENCES`` UPSERT + ``SELECT LAST_INSERT_ID()``로 ``member_id`` 채번 ② ``MEMBERS`` INSERT ③ ``MEMBER_SIGNUP_VERIFICATIONS.consumed_at`` UPDATE — 세 문장 커밋.
- (트랜잭션 밖, 커밋 후) ``MemberSignedUpEvent{memberId, target}`` 발행 — 로그 리스너 1개(``MemberSignedUpEventLogger``)만 구독, 개인정보는 앞 2자만 노출하고 마스킹. 쿠폰 발급 로직은 이 API 범위 밖(별도 프로모션 SR 뒀).

오류 응답

응답 봉투는 모두 `{ "code": "...", "message": "..." }` 형태(MBR-4092만 ``login_url`` 필드 추가, ``MemberRegistrationExceptionHandler``).

코드	HT TP	사유	발생 조건
MBR-4001	400	이름/비밀번호 규칙 위반	<code>`name`</code> 이 비었거나 50자 초과, **또는** <code>`password`</code> 가 8~64자·영문+숫자 포함 규칙 위반 (두 경우 모두 같은 코드로 응답)

MBR-4091	409	인증이 필요합니다	`target` 형식 오류, 또는 STEP 1 코드 검증 실패(오답·만료·미요청·이미 소비/인증됨) — **target 의 가입 여부와 무관하게 항상 이 코드** (존재 오라클 방지)
MBR-4093	409	재발송 필요	STEP 1 진입 시점에 `attempt_count >= 5` (이미 시도 상한 도달, 정답 코드를 내도 거부)
MBR-4092	409	이미 가입된 이메일입니다(+`login_url` 필드)	STEP 1 성공 후 STEP 0 또는 STEP 2에서 `email` 중복 확인 (`uq_members_email`)
MBR-4094	409	이미 가입된 휴대폰번호입니다	STEP 1 성공 후 STEP 0 또는 STEP 2에서 `phone_norm` 중복 확인 (`uq_members_phone_norm`)
MBR-5000	500	일시적인 오류입니다	`DataAccessException` (미분류 PK 충돌 재시도 실패 포함) — 정제된 메시지만 응답, 원본은 서버 로그에만

참조 테이블

`MEMBERS`

`MEMBER\_SIGNUP\_VERIFICATIONS`

`ID\_SEQUENCES`

curl 예시

```
curl -X POST /api/members/signup \
  -H "Content-Type: application/json" \
  -d '{"target": "user@example.com", "code": "123456", "password": "abcd1234", "name": "홍길동", "marketingOptIn": true}'
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-15	SR-295	#1	STEP1/STEP0 로직·조건식은 무변경 — INF-MBR-001의 재발송 리셋으로 선행 데이터 상태가 바뀌어 MBR-4093 회복 경로가 생기고 재발송 후 재시도 시 MBR-4092(+login_url)로 귀결되는 부작용 문서화	shop-api@45e21e8
2026-09-16	SR-298	#1	incrementAttemptCount에 attempt_count<5 조건 추가해 증가+상한판정을 원자 UPDATE로 통합(병렬 버스트 우회 차단). 0행일 때만 selectAttemptCount로 오류코드(4091/4093)만 재분기, 재증가 없음	shop-api@0ed4f7d

INF-MBR-003 로그인

> [반영: FUNC-member-005] 2026-09-12

INF-MBR-003: POST /api/members/login — 로그인

> ****개요:**** 이메일+비밀번호로 로그인해 세션(리프레시 토큰 30일 + API 키)을 발급한다. 5회 연속 실패 시 10분 잠금.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberLoginController.java:32-35`

요청

Method: POST

Path: /api/members/login

Content-Type: application/json

인증: 없음(무인증) — 로그인 전에는 아직 API 키가 없다. `ApiKeyAuthFilter`의 `isOpenRoute` 화이트리스트(`MEMBER\_LOGIN\_PATH`, 정확 일치)에 등록되어 있다(가입 관련 엔드포인트들과 동일한 이유의 예외).

파라미터	위치	타입	필수	설명
email	Body	string	Y	로그인 이메일(정규화는 `trim()`뿐 — 대소문자 동일시는 DB 콜레이션에 의존)
password	Body	string	Y	비밀번호(평문 전송, 서버에서 BCrypt 대조)

응답 (200 OK)

```
{
  "memberId": "M-0053",
  "memberName": "홍길동",
  "grade": "SILVER",
  "apiKey": "mk_1a2b3c4d5e6f...",
  "refreshToken": "5f2c9e10-.....5f2c9e10-.....",
  "refreshTokenExpiresAt": "2026-10-12T10:00:00"
}
```

`memberId`/`memberName`/`grade`: `MEMBERS` 조회 값(`MemberCredential`을 통해 전달,

`passwordHash`는 응답에 절대 포함하지 않는다)

`apiKey`: 이 회원이 이후 `/api/\*\*` 호출에 쓸 키. 최초 로그인이면 신규 발급, 이미 있으면 기존 값 그대로 재 조회(재발급 아님) — `MEMBER\_API\_KEYS.member\_id`가 PK라 회원당 항상 1개

`refreshToken`: 30일 유효, **원문이 노출되는 유일한 응답**. DB에는 SHA-256 해시(소문자 hex)만 저장
(`MEMBER\_REFRESH\_TOKENS.token\_hash`)

`refreshTokenExpiresAt`: `LocalDateTime` 직렬화(ISO-8601 유사 문자열) — 정확한 포맷 단언은 FUNC-
member-004(로그인 화면) 쪽 테스트가 함

비즈니스 규칙

****순서(사람이 직접 확정, 임의 변경 금지)**:** ① 잠금 여부 확인(`MEMBER\_LOGIN\_ATTEMPTS`, 비밀번호 검증 전 — BCrypt 비용 절약) → ② `MEMBERS`를 email로 조회(탈퇴 회원도 포함해서, `del\_yn` 필터 없이) → ③ 비밀번호 대조 → ④ 성공 시 실패카운터 reset → 세션 발급.

****존재 오라클 방지(3경우 완전 동일 응답)**:** "회원 없음" / "탈퇴 회원(`del\_yn='Y'`)" / "비밀번호 불일치" — 이 세 경우는 완전히 동일한 401 `MBR-4011`, 동일한 일반화 문구("이메일 또는 비밀번호가 올바르지 않습니다 (n/5)", 사유를 절대 구분해 노출하지 않음), 동일한 카운터 증가 의미로 응답한다. 회원이 없거나 탈퇴한 경우에도 더미 BCrypt 해시로 `matches()`를 돌려 응답시간을 비슷하게 만든다(타이밍 사이드채널 완화, should).

****잠금 카운터는 회원 존재와 무관**:** `MEMBER\_LOGIN\_ATTEMPTS`는 email 문자열 자체가 PK — 가입되지 않은 이메일도 카운트된다. 이렇게 해야 "이 이메일이 잠겼는가"라는 관찰 자체가 계정 존재의 오라클이 되지 않는다.

****5번째 실패 = 즉시 429(사람 확인 2로 확정)**:** 4번째까지는 401(`n/5` 문구), **5번째 실패는 401이 아니라 곧바로 429 `MBR-4291`**이다. `touchFailure` 한 문장(원자 UPSERT)이 `fail\_count` 증가와 `locked\_until` 세팅을 같은 SET 목록 좌→우 평가로 처리한다(세션변수 미사용). 잠금 만료 후 재실패는 `fail\_count`를 1로 리셋(무한 누적 방지).

****DB 폴백 인증(사람 확인 3으로 확정)**:** `ApiKeyAuthFilter`는 정적 `lab.api-keys` 맵(admin·M-0001만 등록됨) 조회가 실패했을 때만(요청당 최대 1회) `MemberApiKeyDao#selectMemberIdByApiKey`로 DB 폴백 조회한다. 이 폴백은 `MEMBERS` 조인 + `del\_yn='N'` + `revoked\_at IS NULL`로 좁혀져 있어(round9 재작업), 탈퇴 회원이나 폐기된 키는 종전과 동일한 401을 받는다. 기존 정적 맵 경로(admin·M-0001)는 이 추가로 코드·동작이 전혀 바뀌지 않는다(정적 맵 히트 시 DB 조회 자체가 일어나지 않음).

****리프레시 토큰 하우스키핑**:** 로그인 성공마다 새 토큰 INSERT 직후 이 회원의 만료·폐기 행을 지우고(`purgeExpiredOrRevoked`), 활성 토큰이 상한(5개)을 넘으면 `issued\_at` 오래된 순으로 초과분을 지운다(`enforceActiveCap`). 정확한 상한 강제가 아니라 하우스키핑 — 동시 로그인이 6건 이상이면 방금 발급한 토큰이 지워질 수도 있다(사람이 넘긴 low 이슈).

****API 키 동시 최초발급 레이스 방지**:** `issuelfAbsent`는 `member\_id` PK에 대한 no-op UPSERT(`ON DUPLICATE KEY UPDATE member\_id = member\_id`)로, 동시에 여러 로그인 요청이 들어와도 실제로 저장되는 키는 하나뿐이다. 호출부는 이 UPSERT 직후 반드시 `selectByMemberId`로 재조회해야 한다(이 요청이 넘긴 후보 키가 실제로 쓰였다는 보장이 없음).

****`@Transactional` 미사용(사람 결정)**:** 이 클래스는 트랜잭션을 전혀 쓰지 않는다 — 모든 DB 문장을 개별 autocommit 단일 statement로 처리한다(사례집 SR-231 r2 "카운터 증가가 트랜잭션 롤백에 같이 사라짐" 재발 방지).

****레거시/미해결 주의(QA가 지적, 사람이 waiver 수용 — FUNC-006 착수 시 확정 예정)**:**

`MEMBER\_API\_KEYS.revoked\_at`을 세팅하는 운영 코드는 아직 없다(FUNC-006 로그아웃 미구현). FUNC-006이 로그아웃에서 `revoked\_at`을 세팅하기 시작하면, 이 로그인 API의 `issuelfAbsent` (no-op UPSERT, `member\_id`가 PK)는 그 폐기 표시를 지우지 않고 이어지는 `selectByMemberId`도 `revoked\_at` 필터가 없어 죽은 키를 그대로 반환한다 — 로그아웃 후 재로그인한 회원이 200 응답과 함께 이미 무효화된 `apiKey`

를 받는 ****영구 락아웃**** 경로가 남아 있다. 폐기된 키를 되살리거나 새로 교체하는 재발급 경로가 이 API에는 없다. 사람이 확정한 처리 방향(a: 로그인에 폐기 키를 새 키로 교체 / b: 006이 DELETE로 폐기 / c: apiKey 폐기를 006 범위에서 제외)은 FUNC-006 착수 시점에 결정한다(현재는 FUNC-006이 `Approved`·미구현 상태라 즉시 영향 없음).

트랜잭션 순서

1. (비트랜잭션, 읽기) `MEMBER\_LOGIN\_ATTEMPTS`에서 잠금 여부 확인(`selectAttempt`). 잠겨 있으면 즉시 429 응답, 이후 단계 진행 안 함.
2. (비트랜잭션, 읽기) `MEMBERS`를 email로 조회(`selectAuthByEmail`, `del\_yn` 필터 없음 — 탈퇴 회원도 조회해 동일 401로 처리하기 위함).
3. (비트랜잭션) 비밀번호 대조 실패(회원 없음/탈퇴/비번오류) → `touchFailure`(원자 UPSERT)로 실패 카운터 증가(+임계 도달 시 `locked\_until` 세팅) → 401 또는 429 응답, 이후 단계 진행 안 함.
4. (비트랜잭션) 대조 성공 → `reset`(단일 DELETE)으로 실패 카운터 제거.
5. (비트랜잭션) `MEMBER\_REFRESH\_TOKENS` INSERT(신규 토큰) → 같은 회원의 만료·폐기 행 DELETE(`purgeExpiredOrRevoked`) → 활성 토큰 상한 초과분 DELETE(`enforceActiveCap`).
6. (비트랜잭션) `MEMBER\_API\_KEYS` no-op UPSERT(`issuelfAbsent`) → `selectByMemberId`로 재조회 → 200 응답.

사이드이펙트

로그인 성공마다 `MEMBER\_REFRESH\_TOKENS`에서 ****이 회원의**** 만료·폐기 토큰 삭제 + 활성 토큰 5개 초과분 삭제(로그인 자체의 목적인 "신규 토큰 발급" 외의 하우스키핑 삭제).

****공유 보안 필터 변경****: `ApiKeyAuthFilter`에 DB 폴백 조회 경로가 이번 FUNC에서 추가되어, 이 로그인 API가 발급한 `apiKey`로 들어오는 ****다른 모든 `/api/\*\*` 요청****의 인증 판정에 `MemberApiKeyDao` 조회가 새로 개입한다(정적 맵 히트 시에는 개입하지 않음 — 기존 admin-M-0001 경로 무변경).

오류 응답

응답 봉투는 모두 `{ "code": "...", "message": "..." }` 형태(MBR-4291만 `retryAfterSeconds` 필드 추가, `MemberLoginExceptionHandler`).

코드	HT TP	사유	발생 조건
MBR-4011	401	이메일 또는 비밀번호가 올바르지 않습니다 (n/5)	회원 없음 / 탈퇴 회원(`del_yn='Y'`) / 비밀번호 불일치 — **세 경우 모두 동일 코드·문구** (존재 오라클 방지)
MBR-4291	429	로그인 시도 횟수를 초과했습니다. 잠시 후 다시 시도해 주세요(+`retryAfterSeconds`)	`MEMBER_LOGIN_ATTEMPTS.locked_until`이 현재 시각보다 미래(5회 실패 도달, 10분 잠금)
MBR-5000	500	일시적인 오류입니다. 잠시 후 다시 시도해 주세요	`DataAccessException` — 정제된 메시지만 응답, 원본은 서버 로그에만(`MemberLoginExceptionHandler`)

참조 테이블

`MEMBER\_LOGIN\_ATTEMPTS`
`MEMBER\_REFRESH\_TOKENS`
`MEMBER\_API\_KEYS`
`MEMBERS`

curl 예시

```
curl -X POST /api/members/login \  
  -H "Content-Type: application/json" \  
  -d '{"email": "user@example.com", "password": "abcd1234"}'
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커 밋
2026-09-15	QUICK-20260915-1	#1	429/5000 오류 문구를 실제 코드 상수와 일치시킴(잠시 후 다시 시도해 주세요 완곡화)	

INF-MBR-004 로그아웃

> [반영: FUNC-member-006] 2026-09-12

INF-MBR-004: POST

/api/members/sessions/logout — 로그아웃

> ****개요:**** 로그인 세션을 끝낸다 — 이 회원의 API 키와 리프레시 토큰 전체를 폐기한다. 관련: INF-MBR-005(리프레시) — 같은 세션 생명주기의 반대편.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberSessionController.java:39-43`

요청

Method: POST

Path: /api/members/sessions/logout

Content-Type: (바디 없음)

인증: ****필요****. `X-API-Key` 헤더는 `ApiKeyAuthFilter`가 먼저 검증한다(화이트리스트에 없음 — 인증 필요 경로). 컨트롤러는 그 헤더 값을 다시 읽어 어느 회원의 세션인지 서비스가 해석한다(필터가 memberId를 별도로 실어주지 않음).

파라미터	위치	타입	필수	설명
X-API-Key	Header	string	Y	로그인/리프레시로 발급받은 API 키

응답 (204 No Content)

바디 없음. ****항상 204****(idempotent) — `X-API-Key`가 `MEMBER\_API\_KEYS`에서 조회되지 않아도(admin 키, 이미 폐기된 키 등) 에러가 아니라 "이미 로그아웃된 상태와 동일"로 취급해 204를 반환한다.

비즈니스 규칙

****memberId 해석****: `X-API-Key` 값으로 `MemberApiKeyDao#selectMemberIdByApiKey` (로그인/003이 이미 만든 메서드, `MEMBERS.del\_yn='N'` + `revoked\_at IS NULL` 필터 포함)를 조회한다. 못 찾으면(정적 admin 키, 이미 폐기된 키 등) ****에러 없이 idempotent 204 no-op**** — 두 폐기 DAO 호출 자체가 일어나지 않는다.

****트랜잭션(메서드 한정)****: `MemberSessionService#logout`만 `@Transactional`이다(`refresh()`는 비트랜잭션 유지 — 사례집 SR-231 r2 "실패 경로에서 남아야 하는 카운터를 트랜잭션에 넣지 말라" 원칙은 여기 두 폐기에는 적용되지 않는다. 둘 다 함께 성공/실패해야 하는 단일 논리 동작이기 때문).

****두 폐기 순서(사람 확정, round2 재작업)****: `refreshTokenDao.revokeAllForMember` (리프레시 토큰 전체 폐기) → `apiKeyDao.revokeByMemberId` (API 키 폐기) 순으로 실행한다. 트랜잭션이 있어 중간 실패는 롤백

되지만, 혹시 트랜잭션 경계 밖에서 재사용되는 상황까지 대비해 "apiKey가 아직 안 죽었으면 재시도 가능"이 되도록 이 순서로 고정했다(반대 순서였던 round1 구현은 apiKey만 죽고 리프레시 토큰이 살아남아, 그 토큰으로 refresh를 치면 폐기된 apiKey가 새 키로 rotate되어 로그아웃되지 않은 것과 같은 상태로 세션이 되살아나는 결함이 있었다).

****회원 단위 전체 폐기(개별 세션 아님)**:** `MEMBER\_API\_KEYS`가 회원당 1개뿐인 기존 설계와 일관되게, 이 회원의 활성 리프레시 토큰 ****전체****를 폐기한다(디바이스별 개별 로그아웃 모델 아님 — 범위 밖으로 명시적 결정).

****폐기는 DELETE가 아니라 `revoked\_at` 세팅**:** `MEMBER\_API\_KEYS`/`MEMBER\_REFRESH\_TOKENS` 모두 행을 지우지 않고 `revoked\_at`을 채운다 — 재조회 시 조건(`revoked\_at IS NULL`)에서만 제외된다.

****세션 종료 시 장바구니 유지(SR 수용기준)**:** 이 API는 `CART\_ITEMS`를 건드리지 않는다 — 이 요건은 "손대지 않음"으로 충족된다(통합테스트가 로그아웃 전후 장바구니 수량 불변을 단언).

****레거시/미해결 주의(사람이 확인, 후속 SR 후보)**:** `MEMBER\_REFRESH\_TOKENS`에 `member\_id` 인덱스가 없어 `revokeAllForMember`가 풀스캔이다 — 로그아웃이 트랜잭션이 되면서 그 스캔의 락이 커밋 시점까지 유지된다(랩 규모에서는 실패로 이어지지 않는 low 리스크로 QA가 판단, 후속 SR에서 멍등 인덱스 추가 권고).

****레거시 픽스처 주의**:** 정적 맵 회원(M-0001, `lab-member-0001-key`)은 `MEMBER\_API\_KEYS`에 행이 없어 idempotent no-op 분기를 타 204를 반환하지만 실제로 폐기되는 것이 없다(랩 픽스처 특성 — 실 로그인 회원은 전부 DB 발급 키를 쓰므로 실사용 영향 없음).

트랜잭션 순서

`@Transactional`(메서드 한정)

1. `MEMBER\_API\_KEYS`에서 `X-API-Key`로 memberId 조회(`selectMemberIdByApiKey`). 못 찾으면 여기서 종료 — 204(변경 없음).
2. `MEMBER\_REFRESH\_TOKENS` UPDATE — 이 회원의 활성 리프레시 토큰 전체 폐기(`revokeAllForMember`).
3. `MEMBER\_API\_KEYS` UPDATE — 이 회원의 API 키 폐기(`revokeByMemberId`).
4. 커밋 → 204. (2:3 중 실패 시 전체 롤백 — 부분 폐기 상태가 남지 않는다.)

사이드이펙트

`CART\_ITEMS`/`ORDERS` 등 세션 밖 테이블은 건드리지 않는다.

오류 응답

코드	HT TP	사유	발생 조건
(필터, `{"error": "unauthorized"}`)	401	인증 실패	`X-API-Key` 헤더 없음 — `ApiKeyAuthFilter`가 컨트롤러 도달 전 default-deny로 차단(컨트롤러 레벨 `{code,message}` 봉투가 아니라 필터 전역 포맷)

MBR-5000	50 0	일시적인 오류입니다	`DataAccessException` — 정제된 메시지만 응답, 원본은 서버 로그에만 (`MemberSessionExceptionHandler`)
----------	---------	---------------	---

참조 테이블

`MEMBER\_API\_KEYS`

`MEMBER\_REFRESH\_TOKENS`

curl 예시

```
curl -X POST /api/members/sessions/logout \  
-H "X-API-Key: mk_1a2b3c4d5e6f..."
```

INF-MBR-005 자동 로그인 토큰(리프레시)

> [반영: FUNC-member-006] 2026-09-12

INF-MBR-005: POST

/api/members/sessions/refresh — 자동 로그인 토큰(리프레시)

> ****개요:**** 만료 임박한 로그인 정보를 30일 리프레시 토큰으로 자동 재발급한다. 관련: INF-MBR-004(로그아웃) — 같은 세션 생명주기의 반대편.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberSessionController.java:46-49`

요청

Method: POST

Path: /api/members/sessions/refresh

Content-Type: application/json

인증: 없음(무인증) — `ApiKeyAuthFilter`의 `isOpenRoute` 화이트리스트

(`MEMBER\_SESSIONS\_REFRESH\_PATH`, 정확 일치)에 등록. 로그아웃 직후 자동 재로그인 시나리오 등 refresh 시점에는 유효한 API 키가 없을 수 있기 때문(로그인.가입 화이트리스트와 동일한 이유).

파라미터	위치	타입	필수	설명
refreshToken	Body	string	Y	로그인/이전 refresh가 발급한 리프레시 토큰 원문. 서버는 SHA-256(소문자 hex)으로 해시해 대조한다 — 원문은 DB에 저장되지 않는다

응답 (200 OK)

```
{
  "memberId": "M-0053",
  "memberName": "홍길동",
  "grade": "SILVER",
  "apiKey": "mk_1a2b3c4d5e6f...",
  "refreshToken": "5f2c9e10-.....5f2c9e10-.....",
  "refreshTokenExpiresAt": "2026-10-12T10:00:00"
}
```

로그인(INF-MBR-003) `LoginResult`와 ****완전히 동일한 필드명·타입·순서****(계약 — 기존 클라이언트 하위호환 유지)의 `SessionResult` 레코드.

`refreshToken`: 이번 호출에서 새로 회전(rotate)된 신규 토큰 — 요청에 실었던 옛 토큰은 이 응답과 동시에 폐기된다(재사용 불가).

`apiKey`: 직전에 로그아웃 이력이 있어 `revoked\_at`이 세팅돼 있었을 때만 실제로 새 값으로 교체된다. 그렇지 않으면(계속 유효했던 키) 기존 값 그대로.

비즈니스 규칙

****4갈래 완전 동일 401(존재 오라클 방지)****: "토큰 미존재" / "토큰 만료" / "토큰 폐기" / "회원 탈퇴 (`del\_yn='Y'`)" 이 네 경우는 완전히 동일한 401 `MBR-4012`, 동일한 일반화 문구("유효하지 않거나 만료된 로그인 정보입니다", 사유를 절대 구분해 노출하지 않는다) — 로그인(003) "3경우 동일 401" 원칙을 refresh에도 그대로 적용(사례집 SR-231 r5/SR-232 r2 계열 재발 방지).

****자격 검문은 반드시 SQL WHERE에 있어야 함****: `MemberRefreshTokenDao#selectActiveByTokenHash`의 `revoked\_at IS NULL AND expires\_at > now`와 `MemberDao#selectById`의 `del\_yn='N'` — 이 두 필터가 빠지면 로그아웃된 세션이 refresh로 영구히 되살아나는 우회 경로가 생긴다. 애플리케이션 레이어에서 재확인하지 않는다(003의 `del\_yn` 필터 관례와 동일).

****토큰 회전(rotation)****: 성공 시 새 토큰을 INSERT하고, 방금 사용한 구 토큰을 `revokeByTokenHash`로 개별 폐기한다(재사용 방지, 이 회원의 다른 활성 토큰에는 영향 없음). 이어서 003의 하우스키퍼(`purgeExpiredOrRevoked`, `enforceActiveCap` 상한 5)을 그대로 재사용한다 — 신규 배치·신규 컬럼 없음.

****apiKey 조건부 rotate(GATE-005 round2 경계표, 사람 확정)****: `issuelfAbsent`는 no-op UPSERT에서 조건부 rotate로 확장됐다 — 행 없으면 INSERT / `revoked\_at IS NOT NULL` (로그아웃 이력)이면 새 후보 문자열로 교체하고 `revoked\_at`을 NULL로 되돌림 / `revoked\_at IS NULL` (계속 유효)이면 완전 no-op. 폐기된 키 문자열 자체는 절대 재사용하지 않는다. 시그니처는 003 그대로라 로그인 API도 이 SQL을 그대로 쓰지만, 활성 키 경로는 완전 no-op이라 로그인 쪽 회귀는 없다.

****`memberApiKey.xml` UPSERT 대입 순서 의존(구현 주의, 사람이 매퍼 주석에 명시)****: `ON DUPLICATE KEY UPDATE`의 세 대입(`api\_key`/`issued\_at`/`revoked\_at`)은 ****`revoked\_at`이 반드시 마지막****이어야 한다 — MySQL/MariaDB는 SET 절을 좌→우로 평가하며, 앞 컬럼이 이미 갱신된 값을 뒤 컬럼의 `IF(...)`가 보게 되어 있다. 순서가 바뀌면 rotate가 조용히 반쪽(키만 바뀌고 `revoked\_at`은 그대로 남거나 그 반대)이 된다. ****레이트리밋 없음(의도된 설계)****: `MEMBER\_LOGIN\_ATTEMPTS` 같은 실패 잠금이 없다 — refreshToken이 UUID×2라 무차별 대입 표면이 사실상 없다는 판단(과설계 금지, SR-확정문답에 요구 없음).

****세션 만료 시 장바구니 유지(SR 수용기준)****: 이 API는 `CART\_ITEMS`를 건드리지 않는다 — 이 요건은 "손대지 않음"으로 충족된다(통합테스트가 refresh 전후 장바구니 수량 불변을 단언).

****레거시/미해결 주의(사람이 확인, 후속 SR 후보)****: `MEMBER\_REFRESH\_TOKENS`에 `member\_id` 인덱스가 없어 `purgeExpiredOrRevoked`/`enforceActiveCap`이 풀스캔이다(랩 규모에서는 실패로 이어지지 않는 low 리스크로 QA가 판단, 후속 SR에서 먹등 인덱스 추가 권고).

트랜잭션 순서

비트랜잭션, 개별 autocommit statement — 003 관례 유지, 각 statement가 `WHERE ... IS NULL`류로 idempotent해 재시도 안전.

1. 입력 `refreshToken` 원문을 SHA-256(소문자 hex)으로 해시.

2. `MEMBER\_REFRESH\_TOKENS`에서 해시로 활성 토큰 조회(`selectActiveByTokenHash`, `revoked\_at IS NULL AND expires\_at > now`). 없으면 401 `MBR-4012`, 이후 단계 없음.
3. `MEMBERS`를 memberId로 조회(`selectById`, `del\_yn='N'` 필터). 결과 없으면(탈퇴) 401 `MBR-4012`(2와 완전 동일 코드·문구), 이후 단계 없음.
4. `MEMBER\_REFRESH\_TOKENS` INSERT — 새 리프레시 토큰 발급(30일).
5. `MEMBER\_REFRESH\_TOKENS` UPDATE — 방금 쓴 구 토큰 개별 폐기(`revokeByTokenHash`, 회전).
6. `MEMBER\_REFRESH\_TOKENS` DELETE — 이 회원의 만료·폐기 행 정리(`purgeExpiredOrRevoked`, 003 하우스키팅 재사용).
7. `MEMBER\_REFRESH\_TOKENS` DELETE — 활성 토큰 상한(5) 초과분 정리(`enforceActiveCap`, 003 하우스키팅 재사용).
8. `MEMBER\_API\_KEYS` 조건부 rotate UPSERT(`issuelfAbsent`) → `selectByMemberId` 재조회.
9. 200 응답(`SessionResult`).

사이드이펙트

성공마다 003의 하우스키팅(만료·폐기 리프레시 토큰 삭제 + 활성 토큰 5개 초과분 삭제)이 부가로 실행된다(리프레시 발급 자체의 목적 외 정리 작업).

apiKey 조건부 rotate(`issuelfAbsent` 시맨틱 확장)는 **로그인(INF-MBR-003) API가 호출하는 동일 메서드**에도 영향을 준다 — 로그인 쪽 활성 키 경로는 여전히 완전 no-op이라 회귀는 없다(TC-FUNC-member-006-13 실측).

`CART\_ITEMS`/`ORDERS` 등 세션 밖 테이블은 건드리지 않는다.

오류 응답

코드	HT TP	사유	발생 조건
MBR-4012	401	유효하지 않거나 만료된 로그인 정보입니다	토큰 미존재 / 만료 / 폐기 / 회원 탈퇴 — **네 경우 모두 동일 코드·문구** (존재 오라클 방지)
MBR-5000	500	일시적인 오류입니다	`DataAccessException` — 정제된 메시지만 응답, 원본은 서버 로그에만(`MemberSessionExceptionHandler`)

참조 테이블

`MEMBER\_REFRESH\_TOKENS`

`MEMBER\_API\_KEYS`

`MEMBERS`

curl 예시

```
curl -X POST /api/members/sessions/refresh \  
-H "Content-Type: application/json" \  
-d '{"refreshToken": "5f2c9e10-.....5f2c9e10-....."}'
```

INF-MBR-006 비밀번호 재설정 코드 요청

> [반영: FUNC-member-008] 2026-09-13

INF-MBR-006: POST /api/members/password-resets/codes — 비밀번호 재설정 코드 요청

> ****개요:**** 비밀번호를 잊은 회원이 이메일 또는 휴대폰번호를 제출하면 6자리 코드를 생성해 10분간 유효하게 저장하고 "발송"한다(이 랩은 실제 발송 게이트웨이가 없어 로그로 대체). 코드 확인·새 비밀번호 반영은 별도 API(FUNC-member-009, 범위 밖).

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberPasswordResetController.java:39-44`

요청

Method: POST

Path: /api/members/password-resets/codes

Content-Type: application/json

인증: 없음(무인증) — `ApiKeyAuthFilter`의 `isOpenRoute` 화이트리스트

(`MEMBER\_PASSWORD\_RESET\_CODE\_PATH`, 정확 일치)에 등록. 로그인 전(비밀번호를 잊은) 사용자가 호출하므로 아직 API 키가 없기 때문(회원가입·로그인·refresh 화이트리스트와 동일한 이유).

파라미터	위치	타입	필수	설명
target	Body	string	Y	이메일 또는 휴대폰번호. 형식으로 채널을 자동 판별한다(정규식 매칭 실패 시 400). 100자 초과·빈 값도 400

응답 (202 Accepted)

```
{
  "channel": "EMAIL",
  "target": "user@example.com",
  "expiresInSeconds": 600
}
```

`channel`: 판별된 채널 — `EMAIL` 또는 `SMS`.

`target`: 정규화된 값(원문 echo가 아님) — 이메일은 `trim().toLowerCase()`, 휴대폰은 숫자만(하이픈·공백 제거, 단 현재 휴대폰 정규식이 하이픈을 애초에 허용하지 않으므로 이 분기는 사실상 도달하지 않는다).

`expiresInSeconds`: 코드 유효기간(고정값 600 = 10분). 실제로 코드가 갱신됐는지(쿨다운 통과 여부), 회원이 존재하는지는 이 응답만으로 절대 구분할 수 없다(오라클 방지 — 형식 오류가 아닌 한 항상 이 모양의

202).

코드 원문은 이 응답에도, 어떤 로그에도 담기지 않는다(SHA-256 해시만 저장).

비즈니스 규칙

****형식 오류만 400, 그 외는 항상 202(존재 오라클·쿨다운 오라클 방지)****: 빈 값 / 100자 초과 / 이메일·휴대폰 형식 불일치만 400 `MBR-4100`. 회원 존재 여부, 쿨다운(60초 이내 재요청) 위반 여부는 응답에 전혀 드러나지 않는다 — 어느 경우든 동일한 202·`VerificationCodeResult` 바디(사례집 SR-231 "존재 판정 오라클"의 일반화 적용).

****회원 테이블을 구조적으로 조회하지 않음****: `MemberPasswordResetService` 생성자는 `MemberDao`를 주입받지 않는다 — 회원 존재/탈퇴 여부와 무관하게 항상 같은 처리 경로를 타도록 클래스 구조 자체로 강제한다.

****target 정규화·형식 정합****: 이메일은 `trim()+toLowerCase()`, 휴대폰은

`MemberRegistrationService.PHONE\_PATTERN`(`^01[016789][0-9]{7,8}\$`, 하이픈 불허)과 문자 단위로 동일한 정규식을 재사용한다(round2, 형제 가입 API와 판정 불일치를 없애기 위한 사람 수정) — 같은 입력이 재설정·가입 두 API에서 다르게 판정되는 것을 방지. PK(target)가 정규화 값이므로 같은 대상의 다른 표기는 같은 행·같은 쿨다운을 공유한다.

****발송 로그는 재조회 대조 후에만 남긴다(round2, QA FAIL 재작업)****: `touchRequest` UPSERT 직후 `selectByTarget`으로 다시 읽어, 저장된 `code\_hash`가 이번 요청이 만든 해시와 같을 때만(=쿨다운에 걸리지 않고 실제로 반영됐을 때만) "발송" 로그를 남긴다. round1은 이 순서 없이 무조건 로그를 남겨, 쿨다운에 걸린 요청도 로그가 남고 정작 저장된 코드는 이전 것이라 발송 로그와 실제 저장 코드가 불일치하는 결함이 있었다(SR-231 round5 재조회 대조와 동일 기법). 응답은 어느 경우든 무조건 202·상수 바디라 이 판정은 로그 여부에만 쓰이고 오라클을 만들지 않는다.

****`memberPasswordReset.xml` UPSERT SET 순서 의존(구현 주의)****: `ON DUPLICATE KEY UPDATE`의 다섯 대입(`code\_hash`/`expires\_at`/`consumed\_at`/`attempt\_count`/`created\_at`) 중 `created\_at`이 반드시 맨 마지막이어야 한다 — MySQL/MariaDB는 SET을 좌→우로 평가하므로, 앞선 네 IF 조건의 `TIMESTAMPDIFF(SECOND, created\_at, #{now})` 판정이 항상 갱신 전(원본) `created\_at` 값을 보게 하기 위함이다(MEMBER_SIGNUP_RATE_LIMITS round6과 동일 원리). 순서가 바뀌면 쿨다운 판정이 첫 번째 IF에서만 정확하고 이후 컬럼은 이미 갱신된 `created\_at`을 봐서 조용히 깨진다.

****코드값 의미****: `cooldownSeconds=60`(동일 target 재요청 쿨다운), `EXPIRES\_IN\_SECONDS=600`(코드 유효 10분), `CODE\_LENGTH=6`(숫자 코드) — 모두 SR-234 확정 요건.

****레거시/round 이력****: round1은 휴대폰 정규식을 하이픈 허용으로 넓혔다가 형제 가입 API(`MemberRegistrationService`)와 같은 입력에 다른 판정(202 vs 400)을 내는 불일치가 QA에서 지적되어(재작업 지시 low(4)), round2에서 하이픈 불허로 되돌렸다(현재 코드 = round2 상태).

****일일 요청 상한 5회(SR-297 #2)****: 쿨다운 판정과 나란히, target별 일일 상한(`DAILY\_REQUEST\_LIMIT=5`)을 전용 카운터 테이블(`MEMBER\_PASSWORD\_RESET\_RATE\_LIMITS`, SCH-MBR-009)에 대한 원자 UPSERT(`MemberPasswordResetRateLimitDao#touchDailyLimit`)로 판정한다 — 가입

`MemberSignupRateLimitDao#touchRateLimit`과 문자 단위로 동일한 SQL 본문(세션 변수 없이 SET 좌→우 평가만으로 결정적, round6 원리). 이 판정이 코드 테이블 UPSERT(`touchRequest`)보다 ****먼저**** 실행되고, 상한 초과 시 `touchRequest`를 아예 호출하지 않아 발송·코드 갱신 자체를 생략한다(응답은 그래도 항상 동일한 202).

****거부 로그 사유 분기(SR-297 #2, QA CONCERNS round1 재작업)****: 일일 상한 게이트가 거부하면 서버 로그 1줄을 남기되, 재조회한 `dailyCount >= DAILY\_REQUEST\_LIMIT`이면 "일일 상한 초과", 그 외(쿨다운 미

경과)면 "쿨다운"으로 사유를 갈라 적는다(형제 `MemberSignupService#rejectionFor`와 같은 판별 기준).
 `target`은 `mask()`로 가려 원문을 남기지 않는다. 이 로그는 서버 관측용일 뿐 응답에는 전혀 반영되지 않는다(오라클 방지).

****알려진 동작 — 자정 경계(SR-297 #2, STEP 3-0 사람 확인)**:** 자정 직전 요청 뒤 60초 이내에 날짜가 바뀌면, 일일 상한 카운터(SCH-MBR-009, `day\_key` PK)는 새 날짜 행이 생겨 쿨다운 없이 admit(일일 카운트 소비)하지만, 이 코드 테이블의 자체 쿨다운(`created\_at` 기준 60초)은 여전히 걸려 코드가 갱신되지 않을 수 있다 — 일일 카운트만 소비되고 코드는 안 나가는 조합이 가능하다. 응답은 이 경우에도 항상 202·동일 바디라 클라이언트에는 드러나지 않는다(교차-day_key 조회로 이 조합을 막는 것은 범위 밖).

트랜잭션 순서

비트랜잭션, 개별 autocommit statement(`@Transactional` 미사용) — 두
 UPSERT(`touchDailyLimit`·`touchRequest`)가 각각 자체 원자 판정이라 응용 레이어의 트랜잭션이 불필요.
 `MEMBER\_PASSWORD\_RESET\_RATE\_LIMITS`와 `MEMBER\_PASSWORD\_RESETS`는 완전히 분리된 테이블이라
 두 UPSERT 사이에 걸치는 락이 없다.

1. target 형식 검증(빈 값 / 100자 초과 / 이메일·휴대폰 정규식). 실패 시 400 `MBR-4100`, 이후 단계 없음.
2. 채널 판별(`EMAIL`/`SMS`) 및 target 정규화.
3. `now`/`dayKey` 계산, 요청 고유 토큰(UUID) 생성.
4. **** (SR-297 #2) **** `MEMBER\_PASSWORD\_RESET\_RATE\_LIMITS` UPSERT(`touchDailyLimit`) — 쿨다운(60초) AND 일일상한(5회) 둘 다 만족할 때만 `daily\_count`/`last\_requested\_at`/`last\_token`을 갱신 →
 `selectRateLimit` 재조회 → `lastToken`이 내 토큰과 같으면 `admitted`.
5. `admitted=false` (쿨다운 또는 상한 초과)면 ****6~9단계를 건너뛰고 바로 10단계****로 — 거부 사유(재조회한 `dailyCount` ≥ 5면 "일일 상한 초과", 아니면 "쿨다운")를 가려 서버 로그 1줄만 남긴다(target 마스킹).
6. 6자리 코드 생성 → SHA-256(소문자 hex) 해시.
7. `MEMBER\_PASSWORD\_RESETS` UPSERT(`touchRequest`) — 코드 테이블 자체 쿨다운(60초) 미만이면 다섯 컬럼 모두 기존 값 유지(no-op), 이상이면 다섯 컬럼 모두 새 값으로 갱신(`attempt\_count`는 0, `consumed\_at`은 NULL로 리셋).
8. `MEMBER\_PASSWORD\_RESETS`를 target으로 재조회(`selectByTarget`).
9. 재조회한 `code\_hash`가 6단계에서 만든 해시와 같으면(=실제로 반영됐으면) 발송 로그 1줄 기록(코드 원문 미포함, target은 마스킹). 다르면(코드 테이블 쿨다운에 걸려 이전 코드가 그대로 남음 — 자정 경계 등) 로그 생략.
10. 202 응답(`VerificationCodeResult`) — 4~9단계 결과와 무관하게 항상 동일한 모양.

사이드이펙트

`MEMBER\_PASSWORD\_RESETS` (코드 테이블) + `MEMBER\_PASSWORD\_RESET\_RATE\_LIMITS` (SR-297 #2, 일일 상한 카운터 테이블) 두 테이블만 다룬다 — 회원 테이블(`MEMBERS`)은 여전히 조회·변경하지 않는다

(`MemberPasswordResetService` 생성자가 `MemberDao`를 주입받지 않는 구조적 강제).

"발송"은 실제 게이트웨이 호출이 아니라 로그 1줄 기록으로 대체된다(이 랩 환경의 시뮬레이션,

`MemberSignupService`의 가입 인증코드 발송 로그와 동일 모양·수준).

일일 상한 거부 시에도 서버 로그 1줄(사유 포함)을 남긴다(위 "거부 로그 사유 분기" 참고) — 응답에는 반영되지 않는다.

오류 응답

코드	HT TP	사유	발생 조건
MBR-4100	400	이메일 또는 휴대폰번호 형식이 올바르지 않습니다	target이 빈 값 / 100자 초과 / 이메일·휴대폰 정규식 불일치
MBR-5000	500	일시적인 오류입니다	`DataAccessException` — 정제된 메시지만 응답, 원본은 서버 로그에만(`MemberPasswordResetExceptionHandler`)

참조 테이블

`MEMBER\_PASSWORD\_RESETS`

`MEMBER\_PASSWORD\_RESET\_RATE\_LIMITS`(SR-297 #2, SCH-MBR-009)

curl 예시

```
curl -X POST /api/members/password-resets/codes \
  -H "Content-Type: application/json" \
  -d '{"target": "user@example.com"}'
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커밋
2026-09-16	SR-297	#2	일일 상한 5회 게이트(MEMBER_PASSWORD_RESET_RATE_LIMITS UPSERT)를 코드 UPSERT 앞에 배선, 거부 로그 사유 분기, 자정 경계 알려진 동작 기록	shop-api@b7ae32f

INF-MBR-007 비밀번호 재설정 확정

> [반영: FUNC-member-009] 2026-09-13

INF-MBR-007: POST /api/members/password-resets/confirmations — 비밀번호 재설정 확정

> ****개요:**** 이메일/휴대폰으로 받은 6자리 코드와 새 비밀번호를 제출하면 코드를 확인하고, 맞으면 비밀번호를 반영한 뒤 그 회원의 모든 기기를 로그아웃시킨다(리프레시 토큰 + API 키 폐기). 코드 요청은 별도 API(FUNC-member-008, INF-MBR-006)가 담당한다.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberPasswordResetConfirmationController.java:41-45`

요청

Method: POST

Path: /api/members/password-resets/confirmations

Content-Type: application/json

인증: 없음(무인증) — `ApiKeyAuthFilter`의 `isOpenRoute` 화이트리스트

(`MEMBER\_PASSWORD\_RESET\_CONFIRM\_PATH`, 정확 일치)에 등록. 로그인 전(비밀번호를 잊은) 사용자가 호출하므로 아직 API 키가 없기 때문(INF-MBR-006과 동일한 이유). 자격 판정은 필터가 아니라 이 서비스(원자 확정 UPDATE + del_yn 필터 회원 조회)가 한다.

파라미터	위치	타입	필수	설명
target	Body	string	Y	이메일 또는 휴대폰번호. INF-MBR-006이 저장한 것과 같은 정규화 규칙(이메일 trim+소문자, 휴대폰 숫자만)으로 재검증·정규화한다. 형식 오류 시 400
code	Body	string	Y	6자리 인증코드. SHA-256 해시로 저장값과 대조(원문은 저장되지 않음)
newPassword	Body	string	Y	새 비밀번호. 8~64자, 영문+숫자 포함 규칙(가입 API와 동일 검증 재사용). 형식 오류 시 400

응답 (204 No Content)

본문 없음. 아래 어느 경로든(코드 확정 성공 후 회원을 찾은 경우 / 회원을 못 찾은 경우) 동일하게 204를 반환한다 — 회원 존재 여부를 응답으로 절대 구분하지 않는다(`MemberSessionController.logout`)과 동일 형태).

비즈니스 규칙

> 이 섹션은 abstract다. 정본은 앵커(소스)이며, 상세 분기는 file:line을 직접 읽는다.

****존재 오라클 방지(이 API의 핵심 위험)**:** 코드 확정(`confirmIfCodeMatches`)이 회원 조회보다 항상 먼저 실행되고, "행 없음"(요청한 적 없는 target)도 단순 코드 불일치와 완전히 동일한 409 `MBR-4102`로 수렴한다(`handleConfirmFailure`가 두 경우 모두 같은 `mismatchException()` 팩토리 호출). 회원 조회(코드 확정 성공 후 1회) 결과가 있든 없든 응답은 204로 동일 — 이 API의 어떤 응답도 회원 존재 여부를 드러내지 않는다.

****del_yn='N' 이중 필터**:** 회원 매칭 조회(`selectMemberIdByResetTarget`)와 비밀번호 반영 UPDATE(`updatePasswordHash`) 양쪽 모두에 `del\_yn = 'N'`을 건다 — 탈퇴 회원의 비밀번호가 되살아나거나 탈퇴 회원 명의로 세션이 재발급되는 경로를 원천 차단한다.

****코드 확정이 회원 조회보다 항상 먼저**:** `confirmPasswordReset`은 target/newPassword 형식 검증 → 코드 확정(`confirmIfCodeMatches`) → (성공 시에만) BCrypt → 회원 조회 순서로 진행한다. 회원 조회 이전 어떤 단계도 `MEMBERS` 테이블을 참조하지 않는다.

****BCrypt는 확정 성공 시에만, 발견 여부와 무관하게 항상 실행**:** 코드 확정이 성공한 직후 새 비밀번호를 BCrypt로 해싱한다 — 이후 회원을 찾든 못 찾든 이 해싱 자체는 항상 실행됐던 상태이므로, 회원 발견 여부가 처리 시간(타이밍)으로 새지 않는다.

****회원 미발견 시 조용히 204**:** 코드 확정에 성공했지만 매칭되는 활성 회원이 없으면(가짜 target 등) 로그 없이 그대로 반환한다 — 완료 로그는 실제로 비밀번호가 반영된 경우에만 남긴다.

****트랜잭션 경계(Writer 별도 빈)**:** 이 서비스는 `@Transactional`을 쓰지 않는다 — 코드 확정/시도횟수 증가는 각각 autocommit 단일 UPDATE 문(실패 시도 카운터가 트랜잭션 롤백에 함께 사라지는 것을 방지). 비밀번호 반영+전 기기 로그아웃만 별도 스프링 빈(`MemberPasswordResetConfirmationWriter`)의 `@Transactional` 메서드로 묶는다 — 서비스 클래스 안에서 자기 메서드를 직접 호출(self-invocation)하면 Spring 프록시가 트랜잭션 어드바이스를 적용하지 못하는 함정을 피하기 위함이다.

****0행이면 세션 폐기 스킵**:** `updatePasswordHash`가 0행(그 사이 회원 탈퇴 등으로 반영 실효 없음)이면 리프레시 토큰·API 키 폐기 두 문장을 건너뛰고 즉시 반환한다.

****전 기기 로그아웃(사이드이펙트)**:** 비밀번호 반영에 성공하면 그 회원의 리프레시 토큰 전체와 API 키를 폐기한다(순서: 리프레시 먼저, apiKey 나중 — INF-MBR-005 로그아웃과 동일 순서). 이 트랜잭션이 중간에 실패하면(예: 세션 폐기 UPDATE 예외) 비밀번호 반영까지 함께 롤백된다 — "비밀번호는 바뀌었는데 세션은 살아있는" 위험한 중간 상태를 만들지 않는다.

****1회용(consumed_at)**:** `confirmIfCodeMatches`는 `consumed\_at IS NULL`인 행에만 매치하므로, 같은 코드로 두 번째 확정을 시도하면 이미 소비된 것으로 판정되어 410 `MBR-4101`이 된다.

****병렬 버스트 우회 차단(SR-298)**:** `incrementAttemptCount`의 `attempt\_count < 5` 조건절이 단일 PK(`target`) 행 락 위에서 재평가되므로, 동시에 도착한 오답 요청들도 이 한 문장의 짧은 수명 동안만 직렬화된다 — 상한(5)을 넘겨 증가할 수 없다(이전에는 "현재값 읽기 → 상한 미만이면 별도 증가" 순서라 두 문장 사이 레이스로 5를 초과해 증가할 수 있었다). `handleConfirmFailure`는 이 원자 증가를 ****가장 먼저, 무조건**** 시도하고, 1행이면 그 자체로 "미만료·미소비·상한 미만·단순 오답"이 확정되어 `selectByTarget` 없이 바로 409 `MBR-4102`를 던진다. 0행일 때만(상한 도달 하나만이 아니라 이미 소비·이미 만료·행 없음도 0행이다) `selectByTarget`로 원인만 재조회해 4갈래로 분류한다(재증가 없음) — 이 읽기는 경합 판정이 아니라 오류 코드(4101/4102/4103) 표시용일 뿐이다.

트랜잭션 순서

1. target 형식 검증·정규화, newPassword 형식 검증(재사용 — INF-MBR-006/가입 API와 동일 규칙). 실패 시 각각 400, 이후 단계 없음(`attempt\_count` 미소모).

2. 코드 SHA-256 해싱.

3. `MEMBER\_PASSWORD\_RESETS` 원자 확정 UPDATE(`confirmIfCodeMatches`, 비트랜잭션 단일 문, `attempt\_count < 5` 조건 포함). 1행이면 4번으로. 0행이면 `handleConfirmFailure`:

1. 원자 조건부 UPDATE(`incrementAttemptCount`, `attempt\_count < 5` 포함 — SR-298 원자화, read-modify-write 아님)를 **가장 먼저, 무조건** 시도. 1행이면 그 자체로 "미만료·미소비·상한 미만·단순 오답"이 확정 — `selectByTarget` 없이 즉시 409 `MBR-4102`.

2. 0행이면(사유 다양 — 상한 도달/이미 소비·만료/행 없음) `selectByTarget` 읽기 1회로 원인만 재분기(재증가 없음): 행 없음 → 409 `MBR-4102`, 소비·만료 → 410 `MBR-4101`, 상한 도달(`attempt\_count >= 5`) → 409 `MBR-4103`, 그 외(방어용 폴백) → 409 `MBR-4102`.

즉시 응답 종료(이후 단계 진행 안 함).

4. (확정 성공 시에만) BCrypt로 새 비밀번호 해싱.

5. `MEMBERS.selectMemberIdByResetTarget` 조회(이번이 처음 — 이전 어떤 단계도 `MEMBERS`를 조회하지 않음). 못 찾으면 조용히 204(로그 없음).

6. (회원 발견 시) `MemberPasswordResetConfirmationWriter.applyNewPassword`(별도 트랜잭션 빈) 호출:

1. `MEMBERS.updatePasswordHash`(0행이면 이후 두 문장 스킵).

2. `MEMBER\_REFRESH\_TOKENS.revokeAllForMember`(전 기기 로그아웃).

3. `MEMBER\_API\_KEYS.revokeByMemberId`.

7. Writer 트랜잭션 커밋 후에만 완료 로그 1줄(마스킹된 target). 컨트롤러는 204 반환.

사이드이펙트

비밀번호 반영과 함께 그 회원의 전 기기 로그아웃 처리: `MEMBER\_REFRESH\_TOKENS` 전체 폐기 + `MEMBER\_API\_KEYS` 폐기(같은 Writer 트랜잭션, 순서 고정 — 리프레시 먼저, apiKey 나중). `MEMBERS.updated\_at`(V6 마이그레이션 신규 컬럼) 갱신 — 이 FUNC이 최초로 세팅.

오류 응답

코드	HT TP	사유	발생 조건
MBR-4100	400	이메일 또는 휴대폰번호 형식이 올바르지 않습니다	target 형식 검증 실패(재사용 — INF-MBR-006과 동일 코드)
MBR-4001	400	비밀번호 형식이 올바르지 않습니다	newPassword 형식 검증 실패(재사용 — 가입 API와 동일 코드)

MBR-4101	410	인증코드가 만료되었습니다. 다시 요청해 주세요	코드가 이미 소비됐거나(`consumed_at` 있음) 만료됨(`expires_at` 경과) — 신규
MBR-4102	409	코드가 올바르지 않습니다	코드 불일치, 또는 target에 해당하는 행 자체가 없음(둘을 동일 코드로 통일 — 존재 오라클 방지, 신규)
MBR-4103	409	코드 확인 시도 횟수를 초과했습니다. 다시 요청해 주세요	`attempt_count`가 상한(5회) 이상 — 신규
MBR-5000	500	일시적인 오류입니다	`DataAccessException` — 정제된 메시지만 응답, 원본은 서버 로그에만(`MemberPasswordResetConfirmationExceptionHandler`)

참조 테이블

`MEMBER\_PASSWORD\_RESETS`

curl 예시

```
curl -X POST /api/members/password-resets/confirmations \
  -H "Content-Type: application/json" \
  -d '{"target": "user@example.com", "code": "123456", "newPassword": "newPass123"}'
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-16	SR-298	#2	incrementAttemptCount에 attempt_count<5 조건 추가해 증가+상한판정을 원자 UPDATE로 통합(병렬 버스트 우회 차단). handleConfirmFailure는 이 증가를 먼저-무조건 시도하고 0행일 때만 selectByTarget로 원인(행없음/소비·만료/상한) 재분기, 재증가 없음	shop-api@0ed4f7d

INF-MBR-008 배송지 CRUD

> [반영: FUNC-member-011] 2026-09-13

INF-MBR-008: 배송지 CRUD API — 회원 배송지 관리

> ****개요:**** 회원이 자신의 배송지를 목록조회·등록·수정·삭제·기본 배송지 설정하는 API 5종. AS-IS 없음 —

> SR-235(FUNC-member-011)로 신설된 완전 신규 계약이며, 코드가 곧 정보이다.

> ****근거 소스:**** `modules/shop-

api/src/main/java/com/sm/lab/shop/controller/MemberAddressController.java:36-113`

엔드포인트 목록

Method	Path	설명	성공 코드
GET	/api/members/me/addresses	배송지 목록(최근 사용 순)	200
POST	/api/members/me/addresses	배송지 등록	201
PUT	/api/members/me/addresses/{addressId}	배송지 수정	200
DELETE	/api/members/me/addresses/{addressId}	배송지 삭제(소프트)	204
PUT	/api/members/me/addresses/{addressId}/default	기본 배송지로 설정	200

인증·스코프

전 경로 `X-API-Key` 필요(무인증 화이트리스트 아님).

URL에 `memberId`가 없는 "me" 자원이다 — `ApiKeyAuthFilter#evaluateMemberScope`의

`MEMBER\_ME\_ADDRESSES\_PATH`(`^/api/members/me/addresses(?:/.\*)?\$`) 정규식이 5개 경로 전부(메서드

무관)를 `Decision.SCOPE\_TO\_SELF`로 판정한다 — 기존 `GET /api/orders`(memberId 토큰 없는 목록 조회)

강제-자기자신 메커니즘을 재사용한다(새 우회 경로 없음).

admin 키(`\*`)로 호출하면 `evaluateMemberScope` 자체를 타지 않아 이 강제가 일어나지 않는다. 이때

`memberId` 쿼리 파라미터가 비어 있으면 컨트롤러의 `requireMemberId`가 400 `MBR-4200`으로 명시적으로

막는다 — "관리자가 특정 회원 대신 /me를 호출"하는 시나리오는 이 SR에서 정의하지 않는다(사람 확정).

요청

1) GET /api/members/me/addresses — 목록

파라미터	위치	타입	필수	설명
memberId	Query	string	(강제 주입)	클라이언트가 직접 넘기지 않는다 — `ApiKeyAuthFilter`의 SCOPE_TO_SELF가 호출자 자신의 memberId로 강제 덮어쓴다

2) POST /api/members/me/addresses — 등록

파라미터	위치	타입	필수	설명
memberId	Query	string	(강제 주입)	목록과 동일
recipient	Body	string	Y	받는사람 이름, 1~50자
phone	Body	string	Y	휴대폰번호 원문, 1~20자, 정규화 후 `MemberRegistrationService.PHONE_PATTERN` 형식 일치 필요
zipcode	Body	string	Y	우편번호, 숫자 5자리 (`^[0-9]{5}\$`) — ZIPCODES 대조는 이 API 범위 밖 (FUNC-member-012 몫), 클라이언트 값을 그대로 저장
roadAddress	Body	string	Y	도로명 주소, 1~200자
detailAddress	Body	string	Y	상세 주소, 1~200자
entranceMethod	Body	string	N	공동현관 출입방법, 최대 200자
deliveryMemo	Body	string	N	배송 메모, 최대 200자
isDefault	Body	boolean	N	기본 배송지 지정 희망. 첫 등록이면 이 값과 무관하게 무조건 기본 강제

3) PUT /api/members/me/addresses/{addressId} — 수정

파라미터	위치	타입	필수	설명
memberId	Query	string	(강제 주입)	목록과 동일

addressId	Path	long	Y	수정할 배송지 ID
recipient / phone / zipcode / roadAddress / detailAddress / entranceMethod / deliveryMemo	Body	(등록과 동일 타입·검증)	(필수 여부 등록과 동일)	등록과 동일한 검증 규칙 재사용 ('validateFields')
isDefault	Body	boolean	-	**무시됨** — 'MemberAddressService.update'는 'is_default'를 갱신하지 않는다(기본 지정은 전용 API로만)

4) DELETE /api/members/me/addresses/{addressId} — 삭제

파라미터	위치	타입	필수	설명
memberId	Query	string	(강제 주입)	목록과 동일
addressId	Path	long	Y	삭제할 배송지 ID

본문 없음.

5) PUT /api/members/me/addresses/{addressId}/default — 기본 설정

파라미터	위치	타입	필수	설명
memberId	Query	string	(강제 주입)	목록과 동일
addressId	Path	long	Y	기본으로 지정할 배송지 ID

본문 없음.

응답

배송지 객체 공통 셰이프('MemberAddress' — 단건 응답은 봉투 없이 그대로, 목록은 '{items: [...]}' 봉투):

```
{
  "addressId": 12,
  "memberId": "M-0053",
  "recipient": "홍길동",
  "phone": "010-1234-5678",
  "phoneNorm": "01012345678",
  "zipcode": "06236",
  "roadAddress": "서울특별시 강남구 테헤란로 123",
  "detailAddress": "101동 202호",
  "entranceMethod": "공동현관 비밀번호 1234",
  "deliveryMemo": "부재 시 경비실에 맡겨주세요",
}
```

```

"isDefault": "Y",
"lastUsedAt": "2026-09-13T10:00:00",
"createdAt": "2026-09-13T10:00:00",
"updatedAt": "2026-09-13T10:00:00"
}

```

`isDefault`: 이 프로젝트의 Y/N 플래그 관례(`Product.saleYn`과 동일) — boolean이 아니라 문자열 `"Y"/"N"` 그대로 노출된다.

`phoneNorm`: 서버가 계산한 정규화 값(숫자만)도 `MemberAddress` getter에 `@JsonIgnore`가 없어 응답에 그대로 포함된다 — 별도 응답 DTO가 없다(컨트롤러가 도메인 객체를 직접 반환).

`lastUsedAt`은 등록 시 NULL이 아니라 `createdAt`과 동일값으로 채워진다(사람 수정 — 목록/승계 정렬이 NULL 흔들림 없이 안정적으로 동작하게 하기 위함). 이 API 범위에서는 등록 이후 갱신되지 않는다(주문 시 갱신은 SR-255 몫).

목록 GET 응답: `{ "items": [ {..위 세이프..}, ... ] }` — 0건이어도 200 + 빈 배열(404 아님).

등록 POST 응답: 201 + 생성된 배송지 객체(봉투 없음).

수정 PUT 응답: 200 + 수정된 배송지 객체(재조회 결과, 봉투 없음).

삭제 DELETE 응답: 204, 본문 없음.

기본설정 PUT 응답: 200 + 배송지 객체. 이미 기본이면 DB를 건드리지 않고 조회한 현재 상태를 그대로 반환(no-op 맥등).

비즈니스 규칙

> 이 섹션은 "완전한 사양"이 아니라 짧은 abstract다. 정보 진실은 frontmatter `anchors:`의 소스 —

> 여기엔 핵심 분기·코드값 의미만 적는다.

****존재+소유 단일 SQL(존재 오라클 방지)**:** `MemberAddressDao#selectOwned`가 `member\_id`+`address\_id`를

한 WHERE절로 묶어 "배송지 없음"과 "남의 배송지"를 원천적으로 같은 결과(null)로 만든다 → 두 경우

모두 동일 404 `MBR-4041`, 동일 문구. 판정을 분리하는 코드는 없다(수정·삭제·기본설정 3곳 공통, 사례집 SR-231 r5 재발 방지).

****회원당 최대 10개**:** 등록 시 락(`selectByMemberIdForUpdate`)으로 얻은 목록 크기가 10 이상이면 409 `MBR-4201`. 카운트는 락 이후에만 신뢰한다(동시 등록 레이스 방지).

****기본 배송지 자동 승계**:** 삭제 대상이 기본(`is\_default='Y'`)이었으면, 락으로 얻은 목록(대상 제외)

중 `last\_used\_at DESC, created\_at DESC` 1순위가 자동으로 새 기본이 된다. 후보가 없으면(마지막 남은 배송지 삭제) 승계 없이 그대로 기본 없음 상태로 종료.

****첫 등록은 무조건 기본**:** `existing.isEmpty()`이면 요청의 `isDefault` 값과 무관하게 기본으로 강제한다(첫 배송지가 기본이 아닌 상태로 남는 경우를 만들지 않음).

****`last\_used\_at`은 NULL 없이 등록**:** 등록 시 `created\_at`과 동일값으로 채운다 — 목록/승계 정렬 키가 NULL 여부로 흔들리지 않게 하기 위한 사람 확정 결정. 주문 시점 갱신은 이 FUNC 범위 밖(SR-255).

****락 순서 고정(1곳 공통)**:** `register`/`delete`/`setDefault` 세 경로 모두 쓰기 문장 이전에 `selectByMemberIdForUpdate`(`SELECT ... FOR UPDATE`)를 먼저 호출하는 동일 순서를 지킨다 — 두 트랜잭션이 다른 순서로 잡으면 데드락(사례집 SR-231 r3 재발 방지).

`delete`는 이 락 호출 이전에 `selectOwned`로 존재+소유를 먼저 확인한다(싸게 실패). 승계 판정은 락 이후 재조회 없이 이 락 목록(`locked`) 자체를 그대로 쓴다 — MariaDB REPEATABLE READ에서 비잠금 재조회는 락 대기 중 커밋된 변경을 못 볼 수 있어서다(`selectNextDefaultCandidate`는 더 이상 호출되지 않음, 매퍼 테스트용으로만 잔존).

****URL에 memberId 없음**:** `/api/members/me/addresses`는 `ApiKeyAuthFilter`의 `SCOPE\_TO\_SELF` 메커니즘이 memberId를 강제한다(위 "인증:스코프" 절). admin 키로는 400으로 막힌다.

****ZIPCODES 대조는 범위 밖**:** `zipcode`/`roadAddress`/`detailAddress`는 클라이언트가 보낸 값을 검증(형식:길이)만 하고 그대로 저장한다 — 우편번호 실재 여부 대조는 형제 FUNC-member-012 몫.

****수정은 `isDefault`를 다루지 않음**:** 필드 수정과 기본 지정을 분리된 API로 명확히 나눴다 — `update`가 `is\_default`를 건드리면 승계 로직 없이 기본이 조용히 바뀌는 사이드이펙트가 생기므로 의도적으로 배제.

트랜잭션 순서

POST 등록 (`@Transactional`)

1. 휴대폰번호 정규화 + 필드 검증(400, DB 접근 전).
2. `MEMBER\_ADDRESSES` 락 조회(`selectByMemberIdForUpdate`, 첫 문장) — 개수 확인.
3. 10개 이상이면 409, 이후 단계 없음(아직 아무것도 쓰기 전이라 롤백 불필요).
4. 기존 배송지가 있고 이번 등록이 기본이면 `clearDefaultForMember`(기존 기본 해제).
5. `MEMBER\_ADDRESSES` INSERT(`last\_used\_at`=`created\_at`=현재시각, AUTO_INCREMENT로 `addressId` 채번).

DELETE 삭제 (`@Transactional`)

1. `selectOwned`로 존재+소유 확인(없으면 404, 이후 단계 없음).

2. `MEMBER\_ADDRESSES` 락 조회(`selectByMemberIdForUpdate`, register/setDefault와 동일 지점) — 삭제 대상의 `is\_default`와 승계 후보를 이 잠금 목록에서 판정.
3. `MEMBER\_ADDRESSES` UPDATE(`softDelete`, `del\_yn='Y'`).
4. 삭제 대상이 기본이었으면 승계 후보에 `setDefault` UPDATE(후보 없으면 생략).

PUT 기본설정 (@Transactional)

1. `selectOwned`로 존재+소유 확인(없으면 404).
2. 이미 기본이면 즉시 반환(DB 쓰기 없음, no-op 성공).
3. `MEMBER\_ADDRESSES` 락 조회(`selectByMemberIdForUpdate`).
4. `clearDefaultForMember` UPDATE → `setDefault` UPDATE.

PUT 수정 (@Transactional, 단일 UPDATE — 락 불필요)

1. 필드 검증(400).
2. `selectOwned`로 존재+소유 확인(없으면 404).
3. `MEMBER\_ADDRESSES` UPDATE(`updateAddress`, `is\_default` 제외).

GET 목록 (비트랜잭션)

1. `selectList` 단건 SELECT(최근 사용 순, `del\_yn='N'`).

사이드이펙트

DELETE가 대상 배송지의 `is\_default`가 `Y`였을 경우, 메인 목적(삭제) 외에 **다른 행**(승계 후보)의 `is\_default`를 `Y`로 UPDATE한다 — 호출자가 명시적으로 요청하지 않은 추가 쓰기.

POST 등록이 기본으로 강제되는 경우(첫 등록 또는 `isDefault=true`), 기존 기본 배송지 행의 `is\_default`를 `N`으로 먼저 UPDATE한다(`clearDefaultForMember`) — 등록 대상 외 행 변경.

그 외 `CART\_ITEMS`/`ORDERS`/`MEMBERS` 등 이 도메인 밖 테이블은 건드리지 않는다(FK 참조만 존재).

오류 응답

코드	HT TP	사유	발생 조건
MBR-4200	400	유효성 실패	`memberId` 누락(admin 키로 /me 호출 등) / 등록·수정 필드 검증 실패(길이·형식·필수값) — 위반 필드명이 message에 포함됨

MBR-4041	40 4	배송지를 찾을 수 없습니다	수정·삭제·기본설정 대상이 없거나(미존재) 다른 회원 소유(남의 것) — **두 경우 완전 동일 코드·문구**(존재 오라클 방지)
MBR-4201	40 9	배송지는 최대 10개 까지 등록할 수 있습니다	등록 시 락 조회 결과 기존 배송지 수가 10개 이상
MBR-5000	50 0	일시적인 오류입니다	`DataAccessException` — 정제된 메시지만 응답, 원본(경로·SQL·커넥션 정보 포함)은 서버 로그에만(`MemberAddressExceptionHandler`, 로그에도 주소 본문·연락처는 남기지 않음)

참조 테이블

`MEMBER\_ADDRESSES`

curl 예시

1) 목록

```
curl -X GET "/api/members/me/addresses" \
  -H "X-API-Key: <member-key>"
```

2) 등록

```
curl -X POST "/api/members/me/addresses" \
  -H "X-API-Key: <member-key>" \
  -H "Content-Type: application/json" \
  -d '{"recipient":"홍길동","phone":"010-1234-5678","zipcode":"06236","roadAddress":"서울특별시 강남구 테헤란로 123","detailAddress":"101동 202호","entranceMethod":"공동현관 비밀번호 1234","deliveryMemo":"부재시 경비실에 맡겨주세요","isDefault":true}'
```

3) 수정

```
curl -X PUT "/api/members/me/addresses/12" \
  -H "X-API-Key: <member-key>" \
  -H "Content-Type: application/json" \
  -d '{"recipient":"홍길동","phone":"010-1234-5678","zipcode":"06236","roadAddress":"서울특별시 강남구 테헤란로 123","detailAddress":"103동 501호","entranceMethod":"","deliveryMemo":""}'
```

4) 삭제

```
curl -X DELETE "/api/members/me/addresses/12" \
  -H "X-API-Key: <member-key>"
```

5) 기본 설정

```
curl -X PUT "/api/members/me/addresses/12/default" \
  -H "X-API-Key: <member-key>"
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커밋
2026-09-15	SR-235	#2	배송지 CRUD API 구현 — /api/members/me/addresses 목록·등록·수정·삭제·기본 지정 (회원당 최대 10개, 기본 1개, 기본 삭제 시 최근 사용 순 승계), 409 MBR-4201-404 MBR-4041(타인 소유 포함)·400 MBR-4200. v4 시절 완료분이라 이력이 빠져 있어 사후 기록(2026-09-19)	shop-api@1a0cece

INF-MBR-010 api members grades

INF-MBR-010: GET /api/members/grades

> 코드(핸들러) 기준 재동기화 골격(zero-LLM) — 요청·응답·오류 계약은 보강 대상

개요

[TBD]

요청

[TBD]

응답

[TBD]

오류

[TBD]

> 보강: `/sl-sync --apply --kind=inf` 또는 `ddd-api-agent`

INF-ORD-001 회원 목록 조회

INF-ORD-001: GET /api/members/ — 회원 목록 조회

> ****개요:**** 탈퇴하지 않은 회원 전체를 회원ID 순으로 조회한다.

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/MemberController.java:21-25`

요청

Method: GET

Path: /api/members/

Content-Type: application/json

파라미터	위치	타입	필수	설명
(없음)	-	-	-	-

응답 (200 OK)

```
[
  {
    "memberId": "string",
    "memberName": "string",
    "grade": "BRONZE | SILVER | GOLD | VIP",
    "phone": "string",
    "createdAt": "2026-01-01T00:00:00",
    "recentOrders": []
  }
]
```

비즈니스 규칙

`del\_yn` = 'N' 상시필터 → 탈퇴 회원은 목록에서 제외

`recentOrders`는 이 API에서 채워지지 않고 항상 빈 배열([INF-ORD-002])에서만 채워짐)

오류 응답

코드	사유	발생 조건
400	유효성 실패	필수 필드 누락

401	인증 실패	토큰 없음/만료
-----	-------	----------

참조 테이블

테이블	SCH
MEMBERS	[[SCH-ORD-001]]

curl 예시

```
curl -X GET /api/members/ \  
-H "Content-Type: application/json"
```

INF-ORD-002 회원 상세 조회

INF-ORD-002: GET /api/members/{memberId} — 회원 상세 조회

- > ****개요:**** 회원 상세 정보와 함께 최근 주문 5건 요약을 반환한다. (linked_func: FUNC-order-004, LAB-103)
- > ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberController.java:27-31`
- > [반영: FUNC-order-004] SR-207로 신규 화면 `GET /member/{memberId}`(`MemberViewController`)가 이 API의
- > 응답(`MemberService.get` 재사용, HTTP 재호출 없음)을 소비하도록 추가됐다 — 요청/응답/규칙/오류 계약은
- > 변경 없음(AS-IS 유지, qa-agent round2 실측 확인). `screens` 필드는 아직 갱신하지 않았다 — 이 화면의
- > UIS 신규 채번이 아직 안 됐고(STORY-FUNC-order-004 `##` 후속 추적(TODO)` #2.#3), 존재하지 않는
- > UIS-ID를 여기 채우면 참조가 끊긴다. UIS 채번 시 `screens: ["UIS-ORD-00X"]`로 갱신할 것.

요청

Method: GET
Path: /api/members/{memberId}
Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	PathVariable	string	Y	회원 ID

응답 (200 OK)

```
{
  "memberId": "string",
  "memberName": "string",
  "grade": "BRONZE | SILVER | GOLD | VIP",
  "phone": "string",
  "createdAt": "2026-01-01T00:00:00",
  "recentOrders": [
    {
      "orderNo": "string",
```

```

    "orderState": "PLACED | PAID | SHIPPED | PARTIAL_SHIPPED | CANCELED | DONE",
    "totalAmount": 0,
    "orderedAt": "2026-01-01T00:00:00"
  }
]
}

```

비즈니스 규칙

`del\_yn` = 'N' 상시필터 → 탈퇴 회원 조회 시 404

`recentOrders`는 최근 주문 **상위 5건 고정**(`RECENT\_ORDER\_LIMIT`), `ordered\_at` DESC, `order\_no` DESC 순

주문이 없으면 `recentOrders`는 빈 배열(`null` 금지, Member 도메인 필드 기본값)

오류 응답

코드	사유	발생 조건
404	회원 없음	memberId로 조회된 회원이 없거나 탈퇴 회원

참조 테이블

`MEMBERS`

`ORDERS`

curl 예시

```

curl -X GET /api/members/M0001 \
  -H "Content-Type: application/json"

```

INF-ORD-003 주문 목록 조회

INF-ORD-003: GET /api/orders — 주문 목록 조회

> ****개요:**** 회원·주문상태·조회기간으로 필터링한 주문 목록을 페이지징 조회한다. (linked_func: FUNC-order-001, LAB-101)

> [반영: FUNC-order-001] SR-205 — 조회 기간(startDate/endDate) 필터 추가.

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/OrderController.java:28-44`

요청

Method: GET

Path: /api/orders

Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Query	string	N	회원 ID 필터
orderState	Query	string	N	주문상태 필터
startDate	Query	date(ISO-8601)	N	조회 시작일. 미제시 시 종료일 기준 30일 전(오늘-30일)으로 독립 보장. [반영: FUNC-order-001]
endDate	Query	date(ISO-8601)	N	조회 종료일. 미제시 시 오늘로 독립 보장. `startDate`/`endDate`는 각각 독립적으로 기본값이 적용된다(하나만 제시해도 나머지만 보장). [반영: FUNC-order-001]
page	Query	int	N	페이지 번호(기본 1)
size	Query	int	N	페이지 크기(기본 20)

> ****[반영: SR-300]**** "오늘"은 `OrderService`에 주입되는 `Clock`으로 계산한다(운영 기본값 = 시스템 시계 — 결과는 AS-IS와 동일). 계약·기본값 의미는 변경 없음, 테스트에서만 고정 시계로 재현 가능해졌다.

응답 (200 OK)

```
{
  "totalCount": 0,
  "page": 1,
```

```

"items": [
  {
    "orderNo": "string",
    "memberId": "string",
    "memberName": "string",
    "orderState": "PLACED | PAID | SHIPPED | PARTIAL_SHIPPED | CANCELED | DONE",
    "totalAmount": 0,
    "orderedAt": "2026-01-01T00:00:00",
    "items": null,
    "deliveries": null
  }
]
}

```

비즈니스 규칙

`del\_yn` = 'N' 상시 필터 → 논리삭제된 주문 제외

`memberId` 파라미터가 있으면 해당 회원 주문만, `orderState` 파라미터가 있으면 해당 상태만 필터(LAB-101 추가 요구사항)

`offset` = $\max(0, \text{page} - 1) * \text{size}$

`ORDERS`와 `MEMBERS`를 조인해 `memberName`을 함께 반환(회원 미탈퇴 여부와 무관하게 조인만 수행) 목록 응답의 각 항목은 `items`/`deliveries`를 채우지 않음(상세는 [[INF-ORD-004]])

[반영: FUNC-order-001] 기간 필터는 `orderState`/`memberId`와 **AND**로 결합되며,

`ordered\_at`(DATETIME) 기준

양끝 날짜를 포함한다(`ordered\_at` $\geq$ startDate AND `ordered\_at` $<$ endDate + 1일). 응답 스키마 (`totalCount`/`page`/`items[]`)는

변경하지 않는다 — 요청 측 파라미터만 추가.

[반영: FUNC-order-001] `startDate`/`endDate`는 각각 독립적으로 기본값이 적용된 뒤 유효 구간 (`effectiveStart`~`effectiveEnd`)이

역전되면(예: `endDate`만 제시해 보정된 `effectiveStart`가 그보다 늦어지는 경우) 조회를 수행하지 않고 400으로 거부한다.

오류 응답

코드	사유	발생 조건
400	유효성 실패	파라미터 타입 오류(날짜 형식 포함) 또는 유효 조회기간 역전(`effectiveStart` > `effectiveEnd`) [반영: FUNC-order-001]
401	인증 실패	토큰 없음/만료

인증·인가 (SR-204 — 전역 정책)

- > 이 오류 계약은 `/api/\*\*` 전역에 적용되는 횡단 정책이다(구현: `ApiKeyAuthFilter`). 화면
- > 라우트(`/product/\*\*/cart/order/\*\*`)는 무인증 유지. 개별 엔드포인트 INF는 이 정책을 상속한다.

조건	상태	본문
`X-API-Key` 미제시/무효	401	`{"error": "unauthorized"}`
member 스코프 키로 타 회원 자원 접근(소유권 불일치)	403	`{"error": "forbidden"}`
회원 목록·주문 export 등 관리 자원에 member 키	403	admin 키 전용

키 매핑은 앱 설정 `lab.api-keys`(admin→`, member→회원ID). 경로 판정은 서블릿 정규화 경로 기준(퍼센트 인코딩·matrix 파라미터 우회 차단, default-deny). 소유권은 자원의 실제 member_id를 DB 대조(orders/{orderNo}) — 토큰 문자열 대조가 아니다(SR-204 D15).

참조 테이블

테이블	SCH
ORDERS	[[SCH-ORD-004]]
MEMBERS	[[SCH-ORD-001]]

curl 예시

```
curl -X GET "/api/orders?memberId=M0001&orderState=PLACED&page=1&size=20" \
-H "Content-Type: application/json"
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-19	SR-300	#1	'오늘' 계산을 주입 가능한 Clock으로 변경(운영 기본값=시스템 시계, 계약·응답 불변) — anchors 줄번호 OrderService.java:76-90으로 갱신	shop-api@416b260

INF-ORD-004 주문 상세 조회

INF-ORD-004: GET /api/orders/{orderNo} — 주문 상세 조회

> **개요:** 주문 1건을 라인(아이템)·배송 정보를 포함해 조회한다.

> **근거 소스:** `src/main/java/com/sm/lab/shop/controller/OrderController.java:46-50`

요청

Method: GET

Path: /api/orders/{orderNo}

Content-Type: application/json

파라미터	위치	타입	필수	설명
orderNo	PathVariable	string	Y	주문번호

응답 (200 OK)

```
{
  "orderNo": "string",
  "memberId": "string",
  "memberName": "string",
  "orderState": "PLACED | PAID | SHIPPED | PARTIAL_SHIPPED | CANCELED | DONE",
  "totalAmount": 0,
  "orderedAt": "2026-01-01T00:00:00",
  "items": [
    {
      "orderNo": "string",
      "lineNo": 1,
      "sku": "string",
      "productName": "string",
      "qty": 0,
      "unitPrice": 0
    }
  ],
  "deliveries": [
    {
      "deliveryNo": "string",
      "orderNo": "string",
      "deliveryState": "READY | SHIPPED | DELIVERED | CANCELED",
      "invoiceNo": "string",
      "shippedAt": "2026-01-01T00:00:00"
    }
  ]
}
```

```

    }
  ]
}
```

비즈니스 규칙

`del\_yn` = 'N' 상시필터 → 논리삭제된 주문 조회 시 404

`items`는 `ORDER\_ITEMS`와 `PRODUCTS`를 조인해 `productName`을 함께 반환

`deliveries`는 배송이 없으면 빈 배열

오류 응답

코드	사유	발생 조건
404	주문 없음	orderNo로 조회된 주문이 없거나 삭제됨
401	인증 실패	토큰 없음/만료
403	인가 실패	member 스코프 키가 타 회원 주문에 접근

인증·인가 (SR-204 — 전역 정책)

> 이 오류 계약은 `/api/\*\*` 전역에 적용되는 횡단 정책이다(구현: `ApiKeyAuthFilter`). 화면

> 라우트(`/product/\*\*/cart:/order/\*\*`)는 무인증 유지. 개별 엔드포인트 INF는 이 정책을 상속한다.

조건	상태	본문
`X-API-Key` 미제시/무효	401	`{"error": "unauthorized"}`
member 스코프 키로 타 회원 주문 접근(소유권 불일치)	403	`{"error": "forbidden"}`

소유권 판정은 `orderNo`로 `OrderDao.selectByOrderNo`를 조회한 실제 `member\_id`와 요청 키의 스코프를 대조한다(토큰 문자열 대조가 아님, SR-204 D15). admin 스코프(`\*`) 키는 이 필터를 통과해 컨트롤러의 404 분기까지 도달하지만, member 스코프 키는 주문이 없어도(조회 결과 null) 소유권을 확인할 수 없으므로 컨트롤러 도달 전에 403으로 거부된다(즉 member 키로는 존재하지 않는 orderNo도 404가 아니라 403으로 응답됨).

참조 테이블

테이블	SCH
ORDERS	[[SCH-ORD-004]]
MEMBERS	[[SCH-ORD-001]]

ORDER_ITEMS	[[SCH-ORD-003]]
PRODUCTS	[[SCH-ORD-005]]
ORDER_DELIVERY	[[SCH-ORD-002]]

curl 예시

```
curl -X GET /api/orders/20260101-0101 \  
-H "Content-Type: application/json"
```

INF-ORD-005 주문 생성

INF-ORD-005: POST /api/orders — 주문 생성

> ****개요:**** 요청된 상품 라인들을 검증·재고 차감한 뒤 신규 주문을 생성한다.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/OrderController.java:52-56`

요청

Method: POST

Path: /api/orders

Content-Type: application/json

```
{
  "memberId": "string",
  "items": [
    { "sku": "string", "qty": 0 }
  ]
}
```

파라미터	위치	타입	필수	설명
memberId	Body	string	Y	회원 ID
items	Body	array	Y	주문 라인 목록(sku, qty)

응답 (200 OK)

```
{
  "orderNo": "string",
  "memberId": "string",
  "memberName": "string",
  "orderState": "PLACED",
  "totalAmount": 0,
  "orderedAt": "2026-01-01T00:00:00",
  "items": [ { "orderNo": "string", "lineNo": 1, "sku": "string", "productName": "string", "qty": 0,
    "unitPrice": 0 } ],
  "deliveries": []
}
```

비즈니스 규칙

각 라인의 상품이 `sale\_yn = 'Y'` (판매중)가 아니면 400

재고 차감(`decreaseStock`)이 0행이면 재고 부족으로 409 — `WHERE stock\_qty >= qty` 조건부 UPDATE로
동시성 가드

주문번호(`orderNo`)는 `yyyyMMdd-{4자리 시퀀스}` 형식으로 채번(`AtomicInteger`, 시작값 100 — 인스턴스
내 메모리 채번, 재기동 시 초기화)

신규 주문 상태는 항상 `PLACED`

`totalAmount`는 각 라인 `단가 × 수량`의 합

트랜잭션 순서

- 라인별로 `PRODUCTS` 조회 → 판매상태 검증 → 재고 조건부 차감(`UPDATE ... WHERE stock\_qty >= qty`)
- `ORDERS` INSERT (order_state='PLACED')
- 라인별 `ORDER\_ITEMS` INSERT
- 생성된 주문을 [[INF-ORD-004]]와 동일한 조회(`selectByOrderNo` + `selectItems` + `selectDeliveries`)로
재조회하여 반환

오류 응답

코드	사유	발생 조건
400	판매 중인 상품이 아님	상품 미존재 또는 `sale_yn != 'Y'`
409	재고 부족	차감 대상 수량이 현재 재고보다 많음

참조 테이블

테이블	SCH
PRODUCTS	[[SCH-ORD-005]]
ORDERS	[[SCH-ORD-004]]
ORDER_ITEMS	[[SCH-ORD-003]]
MEMBERS	[[SCH-ORD-001]]
ORDER_DELIVERY	[[SCH-ORD-002]]

curl 예시

```
curl -X POST /api/orders \
```

```
-H "Content-Type: application/json" \  
-d '{"memberId":"M0001","items":[{"sku":"SKU001","qty":2}]}'
```

INF-ORD-006 주문 취소

INF-ORD-006: PATCH

/api/orders/{orderNo}/cancel — 주문 취소

> ****개요:**** 주문을 취소 상태로 바꾸고, 이미 취소·완료 상태이거나 출고 완료된 배송이 있으면 거부하며, 취소 성공 시 라인별 재고를 원복한다.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/OrderController.java:59-61`

요청

Method: PATCH

Path: /api/orders/{orderNo}/cancel

Content-Type: application/json

파라미터	위치	타입	필수	설명
orderNo	PathVariable	String	Y	취소할 주문번호

응답 (200 OK)

```
{
  "orderNo": "20260907-0101",
  "memberId": "M0001",
  "memberName": "홍길동",
  "orderState": "CANCELED",
  "totalAmount": 45000,
  "orderedAt": "2026-09-01T10:20:30",
  "items": [
    {
      "orderNo": "20260907-0101",
      "lineNo": 1,
      "sku": "SKU-001",
      "productName": "상품A",
      "qty": 2,
      "unitPrice": 15000
    }
  ],
  "deliveries": [
    {
      "deliveryNo": "D0001",
      "orderNo": "20260907-0101",

```

```

    "deliveryState": "READY",
    "invoiceNo": null,
    "shippedAt": null
  }
]
}

```

취소 완료 후의 주문 상세(items.deliveries 포함)를 그대로 반환한다. `deliveries`가 없는 주문은 빈 배열.

비즈니스 규칙

주문 상태가 `CANCELED` 또는 `DONE`이면 → 409 거부(이미 종결된 주문)

배송 중 하나라도 `SHIPPED` 또는 `DELIVERED` 상태이면 → 409 거부(코드 `ORD-4001`, 출고 완료 배송은 취소 불가)

상태 전이(UPDATE)는 `order\_state NOT IN ('CANCELED','DONE')` 조건부 쿼리로 실행 — 동시 요청 중 하나가 먼저 선점하면 0행이 되어 409로 거부(레이스 가드, 재고는 원복하지 않음)

상태 전이가 성공한 뒤에만 라인별 재고를 원복한다 (linked_func: FUNC-order-002)

트랜잭션 순서

1. `ORDERS` + `MEMBERS` 조회(주문 존재·현재 상태 확인) — 없으면 404
2. `ORDER\_ITEMS` + `PRODUCTS`, `ORDER\_DELIVERY` 조회(라인·배송 상태 확인)
3. `ORDERS.order\_state`를 `CANCELED`로 조건부 UPDATE (미종결 상태일 때만, 레이스 가드)
4. 주문 라인마다 `PRODUCTS.stock\_qty` 증가(재고 원복)
5. `ORDERS` + `MEMBERS`, `ORDER\_ITEMS` + `PRODUCTS`, `ORDER\_DELIVERY` 재조회 후 응답

사이드이펙트

취소 성공 시: 주문에 포함된 각 라인의 `sku`만큼 `PRODUCTS.stock\_qty`를 증가시킨다(메인 목적인 `ORDERS` 상태 변경 외 추가 처리).

오류 응답

코드	사유	발생 조건
404	주문 없음	`orderNo`에 해당하는 주문이 없음
409	취소 불가 상태	주문 상태가 이미 `CANCELED` 또는 `DONE`
409	출고 완료 배송 존재 (ORD-4001)	배송 중 `SHIPPED`/`DELIVERED` 상태가 하나라도 있음
409	취소 불가 상태(동시 처리됨)	상태 전이 UPDATE가 0행 — 동시 요청이 먼저 취소·완료 처리함

참조 테이블

`ORDERS`

`MEMBERS`

`ORDER\_ITEMS`

`PRODUCTS`

`ORDER\_DELIVERY`

curl 예시

```
curl -X PATCH /api/orders/20260907-0101/cancel \  
-H "Content-Type: application/json"
```

INF-ORD-007 주문 배송 목록 조회

INF-ORD-007: GET

/api/orders/{orderNo}/deliveries — 주문 배송 목록 조회

> **개요:** 주문 1건의 배송 목록만 반환한다.

> **근거 소스:** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/OrderController.java:64-68`

요청

Method: GET

Path: /api/orders/{orderNo}/deliveries

Content-Type: application/json

파라미터	위치	타입	필수	설명
orderNo	PathVariable	string	Y	주문번호

응답 (200 OK)

```
[
  {
    "deliveryNo": "string",
    "orderNo": "string",
    "deliveryState": "READY | SHIPPED | DELIVERED | CANCELED",
    "invoiceNo": "string",
    "shippedAt": "2026-01-01T00:00:00"
  }
]
```

비즈니스 규칙

내부적으로 [[INF-ORD-004]]와 동일한 `OrderService.detail()`을 호출한다 — `ORDER\_ITEMS`(`PRODUCTS` 조인 포함)도

함께 조회되지만 응답에는 `deliveries`만 반환된다(불필요한 내부 조회이나 기능상 영향 없음)

배송 목록은 `shipped\_at DESC, delivery\_no DESC`로 정렬(미발송 `READY` 건은 `shipped\_at`이 NULL이라 하단 — LAB-113)

배송이 없으면 빈 배열
오류 응답

코드	사유	발생 조건
404	주문 없음	orderNo로 조회된 주문이 없음

참조 테이블

테이블	SCH
ORDERS	[[SCH-ORD-004]]
MEMBERS	[[SCH-ORD-001]]
ORDER_ITEMS	[[SCH-ORD-003]]
PRODUCTS	[[SCH-ORD-005]]
ORDER_DELIVERY	[[SCH-ORD-002]]

curl 예시

```
curl -X GET /api/orders/20260101-0101/deliveries \
-H "Content-Type: application/json"
```

변경 이력

날짜	변경 내용	변경자
2026-08-23	배송 이력 정렬을 최신순으로(shipped_at DESC, 미발송 null은 하단 · tie는 delivery_no DESC) — LAB-113, linked_func: FUNC-order-006	/sl-change --quick (auto)

INF-ORD-008 판매중 상품 목록 조회

> [변경: SR-201] 2026-08-22

INF-ORD-008: GET /api/products — 판매중 상품 목록 조회

> ****개요:**** 판매중 상태인 상품 전체를 SKU 순으로 조회한다.

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/ProductController.java:24-31`

요청

Method: GET

Path: /api/products

Content-Type: application/json

파라미터	위치	타입	필수	설명
keyword	Query	string	N	상품명 부분 일치 검색어. 미지정 시 전체(판매중) 목록 반환(하위 호환) — [반영: FUNC-order-007] 2026-08-22
inStock	Query	boolean	N	true면 재고 1 이상(`stock_qty >= 1`)인 상품만 반환. 미지정/false는 기존 동작과 동일(하위 호환) — [반영: FUNC-order-007](SR-220) 2026-09-09. non-boolean 값은 Spring 바인딩 실패로 400

응답 (200 OK)

```
[
  {
    "sku": "string",
    "productName": "string",
    "price": 0,
    "stockQty": 0,
    "saleYn": "Y",
    "listPrice": 0,
    "imageUrl": "string"
  }
]
```

> [변경: SR-306] 2026-09-17 — `listPrice`(정가, number|null)·`imageUrl`(대표 이미지 경로, string|null) 필드

> 추가(필드 추가만, 기존 5필드 이름·순서·타입 불변, 하위호환). 값이 없는 상품은 `null`로 내려간다 —

- > 화면은 이때 정가 취소선·할인 배지를 그리지 않는다(할인율은 저장하지 않고 화면이 `price`와 비교 계산).
- > `imageUrl`은 앱이 서빙하는 정적 경로(`/images/products/...`)이며 외부 URL이 아니다.

비즈니스 규칙

`sale\_yn` = 'Y' 상시필터 → 판매중 상품만 반환(단건 조회인 [[INF-ORD-009]]는 이 필터가 없음)

[반영: FUNC-order-007] `keyword` 파라미터가 있으면 `product\_name` 부분 일치(LIKE) 검색 —

`sale\_yn` = 'Y' 상시필터와 AND 결합. `keyword` 미지정 시 기존과 동일하게 전체(판매중) 목록 반환(하위 호환,

기존 소비처 무영향). 응답 스키마(`sku`/`productName`/`price`/`stockQty`/`saleYn`) 변경 없음 —

"품절" 판정은 화면(UIS-ORD-003) 몫으로 `stockQty` 값을 그대로 사용.

[반영: FUNC-order-007](SR-220) `inStock=true`면 `stock\_qty` >= 1` 조건을 `sale_yn` = 'Y' 상시필터와

AND 결합해 품절 상품을 제외한다. `inStock` 미지정 또는 `false`는 기존 동작 그대로(하위 호환).

별도 count 쿼리·페이징이 없어 목록 결과와 총 건수가 항상 일치한다. 응답 스키마 변경 없음.

[반영: SR-306] `listPrice`·`imageUrl`은 표시 전용 — 주문·장바구니 금액 계산은 종전대로 `price`만 쓴다.

인증

> [변경: SR-307] 2026-09-17 — 무키(X-API-Key 헤더 자체 없음) GET 요청은 인증 없이 공개된다
(`ApiKeyAuthFilter`가

> `isOpenRoute` 화이트리스트가 아니라 `doFilterInternal`에서 "무키일 때만" 예외 처리 — 필터는 항상 실행된다).

> **키를 보낸 요청(무효 키 포함)은 이 예외를 타지 않고 종전과 동일한 인증·인가 판정을 그대로 거친다** —

> 예: member 키로 `?memberId=타인`을 보내면 종전과 동일하게 403. GET 외 메서드 (POST/PATCH/DELETE)는

> 이 예외 대상이 아니라 무키 시 401 그대로.

오류 응답

코드	사유	발생 조건
400	유효성 실패	필수 필드 누락
401	인증 실패	키가 있으나 무효, 또는 GET 외 메서드로 무키 요청(위 "인증" 절 참고)

참조 테이블

테이블	SCH
PRODUCTS	[[SCH-ORD-005]]

curl 예시

```
curl -X GET /api/products \
-H "Content-Type: application/json"
```

[반영: FUNC-order-007] `keyword` 검색 예시:

```
curl -X GET "/api/products?keyword=셔츠" \
-H "Content-Type: application/json"
```

[반영: FUNC-order-007](SR-220) `inStock` 필터 예시(품질 제외):

```
curl -X GET "/api/products?inStock=true" \
-H "Content-Type: application/json"
```

변경 이력(이전 기록)

날짜	변경 내용	변경자
2026-08-22	SR-201: keyword 부분일치 검색 파라미터 추가(구현 반영 — FUNC-order-007). path 표기를 실측 라우트(/api/products — 트레일링 슬래시 404)로 정정	sl-aidd STEP 5.5 재동기화
2026-09-09	SR-220: inStock 쿼리 파라미터 추가(재고 1 이상만 조회, 미지정 시 하위호환) — 구현 반영(FUNC-order-007)	sl-aidd STEP 5.5 재동기화
2026-09-17	SR-307: 무키 GET 공개(앱 서빙 /shop 401 해소) — 쿼리·응답 스키마 불변, "인증" 절 신설	sl-aidd STEP 5.5 재동기화

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-17	SR-307	#1	무키 GET 공개(앱 서빙 /shop 401 해소) — 인증 절 신설, 쿼리·응답 스키마 불변	shop-api@cd859a1

2026-09-17	SR-306	#2	응답에 listPrice.imageUrl 필드 추가(필드 추가만, 하위호환, 값 없으면 null)	shop-api, shop-api@7eebc9f, shop-api@e1d18da, shop-web@cbe9816
------------	--------	----	--	--

INF-ORD-009 상품 단건 조회

INF-ORD-009: GET /api/products/{sku} — 상품 단건 조회

> **개요:** SKU로 상품 1건을 조회한다.

> **근거 소스:** `src/main/java/com/sm/lab/shop/controller/ProductController.java:27-31`

요청

Method: GET

Path: /api/products/{sku}

Content-Type: application/json

파라미터	위치	타입	필수	설명
sku	PathVariable	string	Y	상품 SKU

응답 (200 OK)

```
{
  "sku": "string",
  "productName": "string",
  "price": 0,
  "stockQty": 0,
  "saleYn": "Y",
  "listPrice": 0,
  "imageUrl": "string"
}
```

> [변경: SR-306] 2026-09-17 — `listPrice`(정가, number|null)·`imageUrl`(대표 이미지 경로, string|null) 필드

> 추가(필드 추가만, 기존 5필드 이름·순서·타입 불변, 하위호환). 값이 없으면 `null`. [[INF-ORD-008]]과 동일 계약.

비즈니스 규칙

목록 조회([[INF-ORD-008]])와 달리 `sale\_yn` 필터가 없다 — 판매종료 상품도 SKU를 알면 조회 가능
[반영: SR-306] `listPrice`·`imageUrl`은 표시 전용 — 주문·장바구니 금액 계산은 종전대로 `price`만 쓴다.

인증

> [변경: SR-307] 2026-09-17 — [[INF-ORD-008]]과 동일: 무키 GET은 공개(`ApiKeyAuthFilter`
doFilterInternal

> 예외), 키를 보낸 요청은 종전 판정 그대로. **공개 대상 SKU 형식은 `SKU-` 접두사 + 영숫자.하이픈**

> (`^/api/products/(SKU-[A-Za-z0-9-]+)\$`)으로 한정된다 — 이 형식과 맞지 않는 세그먼트(예: 미래에 추가
될

> `/api/products/export` 같은 서브리소스는 이 경로 패턴에 매치되지 않아 무키 401(fail-closed)로 남는다.

오류 응답

코드	사유	발생 조건
401	인증 실패	키가 있으나 무효, 또는 GET 외 메서드로 무키 요청, 또는 sku가 `SKU-` 형식이 아니어서 공개 판정에 매치되지 않음(위 "인증" 절 참고)
404	상품 없음	sku로 조회된 상품이 없음

참조 테이블

테이블	SCH
PRODUCTS	[[SCH-ORD-005]]

curl 예시

```
curl -X GET /api/products/SKU001 \
-H "Content-Type: application/json"
```

변경 이력(이전 기록)

날짜	변경 내용	변경자
2026-09-17	SR-307: 무키 GET 공개(앱 서빙 /shop 401 해소) + 공개 대상 SKU 형식(`SKU-` 접두사) 명시 — 쿼리·응답 스키마 불변, "인증" 절 신설	sl-aidd STEP 5.5 재동기화

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
----	----	----	-------	----

2026-09-17	SR-307	#1	무키 GET 공개 + 공개 대상 SKU 형식(SKU- 접두사) 명시 — 쿼리·응답 스키마 불변	shop-api@cd859a1
2026-09-17	SR-306	#2	응답에 listPrice:imageUrl 필드 추가(필드 추가만, 하위호환, 값 없으면 null)	shop-api, shop-api@7eebc9f, shop-api@e1d18da, shop-web@cbe9816

INF-ORD-010 장바구니 담기

INF-ORD-010: POST /api/cart/items — 장바구니 담기

> ****개요:**** 회원 장바구니에 상품을 담는다. 같은 상품을 재담기하면 기존 수량에 합산된다.

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/CartController.java:22-26`

요청

Method: POST

Path: /api/cart/items

Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Body	string	Y	회원 ID
sku	Body	string	Y	상품 SKU
qty	Body	int	Y	담을 수량 (1 이상)

응답 (200 OK)

```
{
  "memberId": "string",
  "sku": "string",
  "productName": "string",
  "price": 0,
  "qty": 0,
  "addedAt": "2026-08-23T10:00:00",
  "lineTotal": 0
}
```

비즈니스 규칙

같은 상품 재담기는 DB 원자 UPSERT(`INSERT ... ON DUPLICATE KEY UPDATE qty = qty + VALUES(qty)`)로 수량 합산(PK `member\_id,sku`) — 동시 담기에도 lost update-PK 중복 500 없음

판매중지(`sale\_yn='N'`) 상품은 담기 거부(409)

품절(`stock\_qty=0`)은 UPSERT 이전에 조기 거부(409) — 합산·원복 사이클 자체가 불필요

UPSERT 직후 최종(합산) qty가 재고를 초과하면, 방금 더한 만큼만 되돌리고(원복값 0 이하면 행 삭제) 409 — UPSERT가 잡은 행 잠금이 트랜잭션 종료까지 유지되어 판정·원복 사이 다른 트랜잭션 개입 불가

담기는 재고를 차감하지 않는다(보관 전용)

트랜잭션 순서

1. CART_ITEMS UPSERT(수량 합산)
2. CART_ITEMS 병합 결과 SELECT(상품명·현재 단가 조인)
3. (재고 초과 시) CART_ITEMS 원복 — 복원값 > 0이면 UPDATE, 0 이하면 DELETE

오류 응답

코드	사유	발생 조건
400	유효성 실패	qty < 1
404	회원 없음	memberId로 조회된 회원 없음
404	상품 없음	sku로 조회된 상품 없음
409	판매중지 상품	상품의 sale_yn = 'N'
409	품절 상품	상품의 stock_qty = 0
409	재고 초과	합산된 qty가 가용 재고 초과

참조 테이블

테이블	SCH
CART_ITEMS	[[SCH-ORD-006]]
PRODUCTS	[[SCH-ORD-005]]
MEMBERS	[[SCH-ORD-001]]

curl 예시

```
curl -X POST /api/cart/items \
  -H "Content-Type: application/json" \
  -d '{"memberId":"M001","sku":"SKU001","qty":2}'
```

INF-ORD-011 장바구니 조회

INF-ORD-011: GET /api/cart/ — 장바구니 조회

> ****개요:**** 회원의 장바구니 품목 전체와 총액을 조회한다.

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/CartController.java:28-32`

요청

Method: GET

Path: /api/cart/

Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Query	string	Y	회원 ID

응답 (200 OK)

```
{
  "items": [
    {
      "memberId": "string",
      "sku": "string",
      "productName": "string",
      "price": 0,
      "qty": 0,
      "addedAt": "2026-08-23T10:00:00",
      "lineTotal": 0
    }
  ],
  "totalAmount": 0
}
```

비즈니스 규칙

품목은 담은 순(`added\_at`, `sku`)으로 정렬된다

`totalAmount`는 각 품목의 `price × qty`(`lineTotal`) 합계로, 조회 시점에 애플리케이션에서 계산한다(저장 컬럼 아님)

오류 응답

코드	사유	발생 조건
----	----	-------

404	회원 없음	memberId로 조회된 회원 없음
-----	-------	---------------------

참조 테이블

테이블	SCH
CART_ITEMS	[[SCH-ORD-006]]
PRODUCTS	[[SCH-ORD-005]]
MEMBERS	[[SCH-ORD-001]]

curl 예시

```
curl -X GET "/api/cart/?memberId=M001" \  
-H "Content-Type: application/json"
```

INF-ORD-012 장바구니 수량 변경

INF-ORD-012: PATCH /api/cart/items/{sku} — 장바구니 수량 변경

> **개요:** 장바구니에 담긴 품목의 수량을 변경한다.

> **근거 소스:** `src/main/java/com/sm/lab/shop/controller/CartController.java:34-39`

요청

Method: PATCH

Path: /api/cart/items/{sku}

Content-Type: application/json

파라미터	위치	타입	필수	설명
sku	PathVariable	string	Y	상품 SKU
memberId	Query	string	Y	회원 ID
qty	Body	int	Y	변경할 수량 (1 이상)

응답 (200 OK)

```
{
  "memberId": "string",
  "sku": "string",
  "productName": "string",
  "price": 0,
  "qty": 0,
  "addedAt": "2026-08-23T10:00:00",
  "lineTotal": 0
}
```

비즈니스 규칙

qty < 1은 400으로 거부한다 — 삭제는 명시적 DELETE로만 수행(실수 삭제 방지)

요청 qty가 상품 재고를 초과하거나 상품이 품절(stock_qty=0)이면 409

트랜잭션 순서

1. CART_ITEMS UPDATE(qty 변경)

2. CART_ITEMS SELECT(변경 결과 반환용, 상품명·현재 단가 조인)

오류 응답

코드	사유	발생 조건
400	유효성 실패	qty < 1
404	장바구니 품목 없음	memberId+sku 조합의 품목 없음
404	상품 없음	sku로 조회된 상품 없음
409	재고 초과/품절	요청 qty가 가용 재고 초과 또는 상품 품절

참조 테이블

테이블	SCH
CART_ITEMS	[[SCH-ORD-006]]
PRODUCTS	[[SCH-ORD-005]]

curl 예시

```
curl -X PATCH "/api/cart/items/SKU001?memberId=M001" \
  -H "Content-Type: application/json" \
  -d '{"qty":3}'
```

INF-ORD-013 장바구니 품목 삭제

INF-ORD-013: DELETE /api/cart/items/{sku} — 장바구니 품목 삭제

> **개요:** 장바구니에서 품목 1건을 삭제한다.

> **근거 소스:** `src/main/java/com/sm/lab/shop/controller/CartController.java:41-46`

요청

Method: DELETE

Path: /api/cart/items/{sku}

Content-Type: application/json

파라미터	위치	타입	필수	설명
sku	PathVariable	string	Y	상품 SKU
memberId	Query	string	Y	회원 ID

응답 (204 No Content)

응답 본문 없음.

오류 응답

코드	사유	발생 조건
404	장바구니 품목 없음	memberId+sku 조합의 품목 없음

참조 테이블

테이블	SCH
CART_ITEMS	[[SCH-ORD-006]]
PRODUCTS	[[SCH-ORD-005]]

curl 예시

```
curl -X DELETE "/api/cart/items/SKU001?memberId=M001" \
```

```
-H "Content-Type: application/json"
```

INF-ORD-014 장바구니 체크아웃

INF-ORD-014: POST /api/cart/checkout — 장바구니 체크아웃

> ****개요:**** 장바구니 전체 품목을 기존 주문 생성 규칙으로 전환해 주문을 만들고 장바구니를 비운다.

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/CartController.java:53-56`

요청

Method: POST

Path: /api/cart/checkout

Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Body	string	Y	회원 ID

```
{
  "memberId": "M001"
}
```

응답 (200 OK)

```
{
  "orderNo": "20260823-0101",
  "totalAmount": 0,
  "itemCount": 0
}
```

비즈니스 규칙

주문 생성은 새 규칙을 만들지 않고 `OrderService.create`를 그대로 재사용한다(채번·PLACED 상태·재고 차감 — 검증/차감 로직 미복제)

장바구니 조회는 `SELECT ... FOR UPDATE`로 잠근다 — 동일 회원의 동시 체크아웃에서 스냅샷이 아닌 최신 커밋 데이터를 잠그므로, 중복 주문이 발생하지 않는다

잠금 직후 전 품목을 사전 스왑하여 상품없음·판매중지(`sale\_yn='N'`)·재고부족을 한 번에 모아 409 사유 목록 하나로 응답한다(품목별 사유, 부분 주문 없이 전량 거부)

사전 스왑을 통과해도 `OrderService.create`의 실제 재고 차감(`decreaseStock`) 실패가 최종 판정이다(스왑-차감 사이 시점 차 대비)

장바구니는 주문 생성이 트랜잭션 내에서 확정된 뒤에만 비운다(실패 시 품목 보존)

트랜잭션 순서

1. `CART\_ITEMS` 잠금 조회(`FOR UPDATE`) — 비어 있으면 400
2. 품목별 `PRODUCTS` 사전 스왑(상품없음/판매중지/재고부족) — 이슈 있으면 409
3. `OrderService.create` 위임: `PRODUCTS.stock\_qty` 차감 → `ORDERS` 1건 INSERT → `ORDER\_ITEMS` N건 INSERT (동일 트랜잭션 참여)
4. `CART\_ITEMS` 전체 삭제 (주문 확정 후)

사이드이펙트

체크아웃 성공 시 품목별로 `PRODUCTS.stock\_qty`가 차감된다(`OrderService.create` → `decreaseStock`)

체크아웃 성공 시 `CART\_ITEMS`에서 해당 회원의 품목이 전부 삭제된다(`deleteAllItems`)

오류 응답

코드	사유	발생 조건
400	빈 장바구니	memberId의 장바구니에 품목 없음
400	판매 중인 상품이 아님	`OrderService.create` 내부 판정 — 사전 스왑 통과 후 시점 차로 발생 가능
404	회원 없음	memberId로 조회된 회원 없음
409	재고 부족/판매중지/상품없음	사전 스왑에서 품목별 사유 목록으로 응답(가용재고/요청수량 포함)
409	재고 부족(차감 실패)	`OrderService.create`의 `decreaseStock` 0건 — 스왑 이후 경합

참조 테이블

`CART\_ITEMS`

`MEMBERS`

`PRODUCTS`

`ORDERS`

`ORDER\_ITEMS`

curl 예시

```
curl -X POST /api/cart/checkout \
  -H "Content-Type: application/json" \
  -d '{"memberId": "M001"}'
```

INF-ORD-015 주문 목록 CSV 내보내기

INF-ORD-015: GET /api/orders/export — 주문 목록 CSV 내보내기

> ****개요:**** [[INF-ORD-003]]과 동일한 필터·정렬·상시필터로 조회한 주문 목록을 CSV 파일로 내려받는다.
(linked_func: FUNC-order-013, LAB-104)

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/OrderController.java:79-90`

요청

Method: GET

Path: /api/orders/export

Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Query	string	N	회원 ID 필터
orderState	Query	string	N	주문상태 필터(정분)
status	Query	string	N	주문상태 필터 별칭 — `orderState`와 동시에 오면 `orderState` 우선(D11, QA r1 FAIL 정정)

응답 (200 OK)

`Content-Type: text/csv; charset=UTF-8`, `Content-Disposition: attachment; filename="orders\_{yyyyMMdd\_HH:mm:ss}.csv"`

UTF-8 BOM(`EF BB BF`)으로 시작하는 CSV 바이너리. 헤더 행 + [[INF-ORD-003]] 목록 조회 결과를 행 단위로 직렬화(페이징 미적용 — 전체 건).

```
주문번호,회원,상태,금액,주문일시
2026082300001-0101,M0001,PLACED,15000,2026-08-23T10:00:00
```

> ****헤더 라벨(SR-223)**:** 데이터 컬럼 순서·필드명·타입·의미
(`orderNo,memberId,status,totalAmount,orderedAt`)는

> 그대로다. 헤더 텍스트만 같은 위치의 한글 라벨로 1:1 치환했다(주문번호↔orderNo, 회원↔memberId,

- > 상태↔status, 금액↔totalAmount, 주문일시↔orderedAt). 근거: `docs/변경관리/SR-223/00\_요구사항.md`
- > 확정 문답 `header\_label\_order`.

비즈니스 규칙

조회 규칙은 [[INF-ORD-003]]과 완전히 동일(`OrderDao.selectOrders` 재사용) — 별도 조회 규칙 없음.

`del\_yn` = 'N' 상시필터 포함

페이징 미적용: `offset=0`, `size=Integer.MAX\_VALUE`로 고정 호출(전체 건 CSV화)

`orderedAt`은 `ISO\_LOCAL\_DATE\_TIME` 고정 패턴으로 직렬화(가변폭 금지)

CSV 필드는 RFC 4180 인용 규칙 적용(콤마·큰따옴표·개행 포함 시 큰따옴표로 감싸고 내부 큰따옴표는 이중화)

CSV 수식 인젝션 방어: 필드 선두 문자가 `=`,`+`,`-`,`@`이면 작은따옴표(``)를 앞에 붙여 Excel이 수식으로 해석하지 않게 함(D11)

파일 본문은 UTF-8 BOM 프리픽스 포함(Excel에서 한글 깨짐 방지)

오류 응답

코드	사유	발생 조건
400	유효성 실패	파라미터 타입 오류
401	인증 실패	토큰 없음/만료

인증·인가 (SR-204 — 전역 정책)

- > 이 오류 계약은 `/api/\*\*` 전역에 적용되는 횡단 정책이다(구현: `ApiKeyAuthFilter`). 화면
- > 라우트(`/product/\*\*/cart/order/\*\*`)는 무인증 유지. 개별 엔드포인트 INF는 이 정책을 상속한다.

조건	상태	본문
`X-API-Key` 미제시/무효	401	`{"error": "unauthorized"}`
member 스코프 키로 타 회원 자원 접근(소유권 불일치)	403	`{"error": "forbidden"}`
회원 목록·주문 export 등 관리 자원에 member 키	403	admin 키 전용

키 매핑은 앱 설정 `lab.api-keys` (admin→`, member→회원ID). 경로 판정은 서블릿 정규화 경로

기준(퍼센트 인코딩·matrix 파라미터 우회 차단, default-deny). 소유권은 자원의 실제 member_id를

DB 대조(orders/{orderNo}) — 토큰 문자열 대조가 아니다(SR-204 D15).

반출 상한 (SR-204 R-3)

결과 1000행 초과 → **413** `{"error": "payload\_too\_large", "limit": 1000}` (부분 반출 위장 없이 전량 거부, CSV 조립 전 판정). 이 엔드포인트는 **admin 키 전용**.

참조 테이블

ORDERS

MEMBERS

curl 예시

```
curl -X GET "/api/orders/export?memberId=M0001&orderState=PLACED" \  
-o orders.csv
```

INF-ORD-016 api zipcodes

INF-ORD-016: GET /api/zipcodes

> 코드(핸들러) 기준 재동기화 골격(zero-LLM) — 요청·응답·오류 계약은 보장 대상

개요

[TBD]

요청

[TBD]

응답

[TBD]

오류

[TBD]

> 보장: `sl-sync --apply --kind=inf` 또는 ddd-api-agent

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커밋
2026-09-19	SR-235	#3	GET /api/zipcodes 우편번호(도로명) 검색 API 구현 — 200 {items: [{zipcode,roadAddress,sido,sigungu}]}(0건 포함), 400 MBR-4202(검색어 정규화 후 2~50자 범위 밖), 숫자 검색어는 zipcode 전방일치도 포함(OR)	shop-api@1c82e8c, shop-web@8e1e3e3

INF-ORD-017 shop spa static serving

INF-ORD-017: GET /shop/** — 쇼핑 SPA 정적 서빙 + 딥링크 폴백

- > ****개요:**** shop-web(React/Vite) 빌드 산출물(`dist`)을 shop-api가 `/shop/\*\*` 경로로 직접 서빙한다.
- > 별도 개발 서버(:5273) 재기동 없이, shop-api 재기동 한 번으로 화면 변경이 반영되게 하는 것이 목적 (SR-301).
- > 새 REST 엔드포인트가 아니라 정적 리소스 매핑 + SPA 딥링크 폴백이며, 새 오류 코드·인증 계약 변경은 없다.
- > ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/web/ShopStaticResourceConfig.java:75-95`,
`modules/shop-api/src/main/java/com/sm/lab/shop/controller/ShopIndexController.java:46-56`

요청

Method: GET

Path 패턴 2종(경로 특이도로 자동 우선순위 결정, 등록 순서 무관):

`/shop/assets/\*\*` — Vite 빌드 산출물 중 해시 붙은 JS/CSS(장기 캐시 대상)

`/shop/\*\*` — 그 외 전부(정적 파일이 실재하면 그 파일, 없으면 SPA 딥링크로 간주)

`/shop`·`/shop/` (경로 세그먼트 없음, 정확 일치) — `ShopIndexController`가 전담(아래 "구현 세부" 참고)

인증: 무인증(`ApiKeyAuthFilter.isOpenRoute`에 `/shop`·`/shop/\*\*` 화이트리스트, `modules/shop-api/.../ApiKeyAuthFilter.java:230`). 실제 데이터 접근은 여전히 `/api/\*\*` (불변)에서만 일어나므로 이 화이트리스트가 자격 판정을 우회하지 않는다.

응답

요청 형태	응답
`/shop/assets/{해시파일}` (실재)	200, 원본 Content-Type, `Cache-Control: max-age=31536000, public, immutable`
`/shop`, `/shop/`, `/shop/index.html`	200, `text/html`, `Cache-Control: no-cache`, index.html 본문
`/shop/{미매칭 딥링크}` (예: `/shop/products/123`)	200, `text/html`, `Cache-Control: no-cache`, index.html 본문 (SPA 클라이언트 라우팅 위임)
빌드 산출물 자체가 없음(`index.html` 부재)	404(Spring 표준, `NoResourceFoundException` 계열) — 200으로 위장하지 않는다
`/shop/assets/{존재하지 않는 파일}`	404(자산은 폴백하지 않는다 — MIME 오류를 index.html로 감추지 않는다)

비즈니스 규칙

정적 자산(`/shop/assets/\*\*`)과 SPA 폴백(`/shop/\*\*`)은 서로 다른 캐시 정책을 쓴다: 해시 붙은 자산은 불변으로 간주(1년, `immutable`), `index.html`은 배포 직후 옛 화면이 남지 않도록 `no-cache`.

SPA 폴백은 `PathResourceResolver`를 확장해 `super.getResource()`(Spring 내장 위치-이탈 방지)로 실제 파일을 먼저 찾고, 없을 때만 `index.html`로 대체한다 — 새 경로 순회(traversal) 표면을 만들지 않는다.

`ResourceHttpRequestHandler`는 매핑 패턴을 벗겨낸 나머지 경로가 빈 문자열이면(정확히 `/shop`·`/shop/`) 리졸버 체인 진입 전에 자체 가드로 404를 내는 실측 제약이 있다 — 이 두 정확 경로만 `ShopIndexController`(`@GetMapping("/{shop}"/"shop/")`)가 별도로 `index.html`을 직접 읽어 반환한다. 하위 미매칭 경로(`/shop/products/123` 등)는 리소스 경로가 비어있지 않으므로 `ShopStaticResourceConfig`의 폴백 리졸버가 정상 처리한다.

빌드 산출물이 없어도 shop-api 기동은 성공한다 — 기동 시 1회(`@PostConstruct`) `index.html` 존재를 확인해 없으면 WARN 로그("...해결: modules/shop-web에서 npm run build 실행 후 shop-api 재빌드·재기동.")를 남기고, 요청마다 반복 로깅하지 않는다.

`shop.static-location` 프로퍼티(기본값 `classpath:/static/shop/`)는 후행 슬래시 유무와 무관하게 정규화되어 동작한다(`ShopStaticResourceConfig`·`ShopIndexController` 양쪽 생성자에서 동일하게 보정).

`/shop`으로 시작하지만 `/shop/`이 아닌 경로(예: `/shopkeeper`)는 화이트리스트 오매칭 없이 여전히 401(`ApiKeyAuthFilter`가 `equals("/shop") || startsWith("/shop/")` 정확 조합 사용).

****클라이언트 진입 규칙(SR-302.1, 서버 계약 불변)**:** 이 엔드포인트가 `index.html`을 반환한 뒤, 그 안의 SPA가 해시 없이 최초 로드되면 쇼핑 홈(`UIS-ORD-008`)으로 1회 이동하고 이후 `#/`는 주문 목록이다 — 자세한 내용은 `UIS-ORD-008`의 "진입/라우팅" 참고. 이 문서의 요청·응답·앵커 줄 범위는 변경되지 않았다(클라이언트 라우팅은 이 서버 엔드포인트 밖의 동작).

빌드·배포 절차 (AC "빌드 산출물 위치·갱신 방법" 확정값)

1. `modules/shop-web`에서 `npm run build` 실행 → `modules/shop-web/dist` 생성. 이때 `vite.config.ts`의 `base`가 `command==='build'`일 때만 `/shop/`으로 설정되어(개발 서버 `npm run dev`, :5273은 `base:/'` 그대로 무변경) 산출물의 자산 참조 경로가 `/shop/assets/...`로 나온다.

2. `modules/shop-api`에서 Maven 빌드(`mvnw` 등) 시 `generate-resources` 단계에서 `maven-resources-plugin`(`pom.xml:93-119`, 실행 id `copy-shop-spa-dist`)이 `./shop-web/dist`를 `target/classes/static/shop`으로 복사한다. `src/main/resources`(소스 트리)는 건드리지 않으므로 `.gitignore` 변경 불필요.

3. `shop-web/dist`가 없는 환경(예: 아직 한 번도 빌드하지 않음)에서도 이 복사 단계는 조용히 건너뛰고 빌드·기동이 실패하지 않는다.

4. 화면을 갱신하려면: shop-web에서 `npm run build` 재실행 → shop-api 재빌드·재기동 1회. (SR-301의 목적 — 별도 개발 서버 재기동 불필요)

오류 응답

새 오류 코드는 만들지 않는다(`{code,message}` 오류 계약 대상 아님 — 정적 리소스 표준 응답만 사용).

상태	조건
----	----

404	요청 경로에 대응하는 정적 파일 `index.html` 모두 없음(빌드 산출물 없음)
404	`/shop/assets/**` 하위 자산 요청인데 실제 파일 없음(SPA 폴백 대상 아님)
401	`/shop`~`/shop/**` 밖의 경로(`/api/**` 등)를 인증 헤더 없이 요청(기존 계약 불변)

범위 밖

빌드된 SPA가 실제로 `/api/\*\*`를 호출해 정상 동작하는지(런타임 API 연동)는 이 SR 범위 밖 — 후속 SR 후보.

`npm run build` + Maven 빌드를 한 명령으로 묶는 배포 자동화는 이 SR 범위 밖.

개발 서버(:5273)의 `server.proxy` 등 구성 변경 없음 — `vite.config.ts`는 `base`(빌드 전용) 한 줄만 조건부 변경.

스펙 ID 참고

`INF-ORD-017`은 SR-212(`docs/00\_FUNC/stories/STORY-FUNC-order-016.md`, "상품 리뷰 API")가 과거 예약했던 것과 같은 번호이나, SR-212는 원장에서 SUPERSEDED(대체)되어 그 예약이 살아있는 점유가 아니고 (claim 해제됨, v5.1부터 실행 불가한 레거시 FUNC STORY로 읽기 전용 이력), 실제 `INF-ORD-017.md` 스펙 파일 자체가 이 SR 이전까지 존재하지 않았다(사람 확인·승인 — SR-301.1 STORY `## Dev 기록`).

curl 예시

```
curl -i http://localhost:8087/shop/
curl -i http://localhost:8087/shop/products/123
curl -i http://localhost:8087/shop/assets/index-9yozd6aq.js
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-17	SR-301	#1	신규: shop-web 빌드 산출물을 /shop/**로 정적 서빙 + SPA 딥링크 폴백 (ShopStaticResourceConfig·ShopIndexController), /shop 인증 화이트리스트, maven-resources-plugin dist 복사, vite base 조건부 변경	shop-api@0f75df7, shop-web@7feb578
2026-09-17	SR-302	#1	클라이언트 진입 규칙 각주 추가(서버 계약·앵커 불변) — index.html 반환 후 SPA가 쇼핑 홈으로 이동하는 규칙을 UIS-ORD-008에 위임	shop-web@efd0e7c